

Uniwersytet Mikołaja Kopernika
w Toruniu
Wydział Matematyki i Informatyki
Kierunek Informatyka

**Programowanie równoległe w języku Chapel
na przykładzie problemu rozchodzenia się ciepła**

Szymon Orzechowski

Promotor: dr. Marek Nowicki

Spis treści

1	Wstęp	3
2	Problem i wykorzystane technologie	4
2.1	Równanie ciepła i metoda różnic skończonych	4
2.1.1	Model fizyczny	4
2.1.2	Warunki początkowe i brzegowe	4
2.1.3	Dyskretyzacja: jawna metoda różnic skończonych	5
2.1.4	Stabilność numeryczna i warunek CFL	6
2.2	Zrównoleglenie obliczeń stencilowych	8
2.2.1	Dekompozycja dziedziny i komórki brzegowe	8
2.2.2	Wymiana brzegów jako jedyna komunikacja	8
2.2.3	Intensywność arytmetyczna i ograniczenia wydajności	10
2.2.4	Miary skalowalności	10
2.3	Chapel, PGAS i GASNet	11
2.3.1	Język Chapel i model PGAS	11
2.3.2	Iteratory równoległe i tablice globalne	12
2.3.3	Konstrukcje równoległości i synchronizacja	13
2.3.4	Warstwa zadań qthreads oraz warstwa komunikacji GASNet	14
2.3.5	Konduity GASNet	15
3	Klaster i implementacja	17
3.1	Architektura klastra i środowisko	17
3.1.1	Sprzęt i topologia	17
3.1.2	Konfiguracja kompilatora Chapel	18
3.1.3	Dwa kanały komunikacyjne GASNet	18
3.1.4	Uruchamianie i samoopisujące się logi	19
3.1.5	Procedura wdrożenia krok po kroku	20
3.1.6	Uruchamianie eksperymentu	21
3.2	Struktura programu i krok obliczeniowy	23
3.2.1	Dziedzina rozproszona z otoczką	23

3.2.2	Pojedynczy krok obliczeniowy	23
3.2.3	Pomiar czasu i rejestrowanie	24
3.2.4	Parametry konfiguracyjne	25
3.2.5	Rejestrowanie przebiegów	25
3.3	Buforowanie, zapis i agregacja danych	26
3.3.1	Zapis fragmentów na dysk lokalny	26
3.3.2	Agregacja rozproszonych fragmentów	27
3.3.3	Wizualizacja	27
4	Metodologia badań	29
4.1	Scenariusze i metryki	29
4.1.1	Scenariusze badawcze	29
4.1.2	Metryki	30
4.2	Procedura i narzędzia pomiarowe	31
4.2.1	Procedura pomiarowa	31
4.2.2	Narzędzia pomiarowe	32
4.2.3	Przykład wyznaczenia metryk	33
4.2.4	Kontrola warunków i zagrożenia dla trafności	33
5	Wyniki i analiza	34
5.1	Poprawność i wydajność wariantów	34
5.1.1	Weryfikacja poprawności numerycznej	34
5.1.2	Skalowanie wątkowe na pojedynczym węźle	35
5.2	Skalowanie i ograniczenia komunikacji	36
5.2.1	Skalowanie liczbą węzłów	37
5.2.2	Skalowanie wątkowe przy dziewięciu węzłach	38
5.2.3	Skalowanie rozmiaru zadania	40
5.2.4	Synteza: system ograniczony komunikacją	41
5.2.5	Zestawienie zbiorcze i powtarzalność	42
6	Podsumowanie i wnioski	44
6.1	Wnioski	44
6.2	Ograniczenia	45
6.2.1	Kierunki dalszych prac	45
	Bibliografia	47

Rozdział 1

Wstęp

Symulacje zjawisk opisanych równaniami różniczkowymi cząstkowymi należą do najczęściej spotykanych zadań obliczeniowych w nauce i inżynierii. Wśród nich szczególne miejsce zajmują obliczenia *stencilowe*, czyli takie, w których nowa wartość w każdym punkcie siatki jest funkcją wartości z punktu sąsiadującego. Wzorzec ten sprawia, że wydajne i skalowalne wykonanie obliczeń stencilowych na współczesnych architekturach równoległych [16].

Trójwymiarowe równanie przewodnictwa ciepła jest tradycyjnym przykładem takiego problemu. Jego jawna dyskretyzacja różnicowa ma dokładnie te cechy, które sprawiają, że obliczenia stencilowe są trudne do zrównoleglenia: niską intensywność arytmetyczną oraz konieczność wymiany danych brzegowych między częściami dziedziny w każdym kroku czasowym. Program symulujący równanie ciepła nadaje się dobrze do badania granic skalowalności.

Równoległe programowanie systemów rozproszonych tradycyjnie realizuje się za pomocą interfejsu przekazywania komunikatów (MPI), co daje wysoką wydajność kosztem znacznej złożoności kodu: programista jawnie zarządza podziałem danych, buforami wymiany oraz synchronizacją [11]. Alternatywę oferują języki i biblioteki oparte na modelu *PGAS* (ang. *Partitioned Global Address Space*), w którym pamięć całego systemu jest widziana jako jedna, logicznie spójna, lecz fizycznie partycjonowana przestrzeń adresowa [8]. Język Chapel, rozwijany pierwotnie przez firmę Cray, jest jednym z bardziej rozwiniętych języków w tej grupie [1]. Umożliwia zapisywanie obliczeń rozproszonych w notacji zbliżonej do sekwencyjnej: z globalnym widokiem tablic, wysokopoziomowymi iteratorami równoległymi i gotowymi rozkładami dziedzin. Szczegóły komunikacji pozostawia warstwie wykonawczej, opartej na bibliotece GASNet.

Celem pracy jest zaprojektowanie, zaimplementowanie i wdrożenie na klastrze rozproszonego programu symulującego równanie przewodnictwa ciepła w Chapel oraz analizę jego poprawności i skalowalności.

Rozdział 2

Problem i wykorzystane technologie

2.1 Równanie ciepła i metoda różnic skończonych

2.1.1 Model fizyczny

Rozważanym zjawiskiem fizycznym jest dyfuzja ciepła w jednorodnym, izotropowym ośrodku wypełniającym sześcian jednostkowy $\Omega = [0, 1]^3$. Pole skalarne $u(\mathbf{x}, t)$ opisuje temperaturę w punkcie $\mathbf{x} = (x, y, z) \in \Omega$ w chwili $t \geq 0$. Ewolucję tego pola opisuje równanie różniczkowe cząstkowe znane jako równanie przewodnictwa ciepła, zwane też równaniem dyfuzji:

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right), \quad (2.1)$$

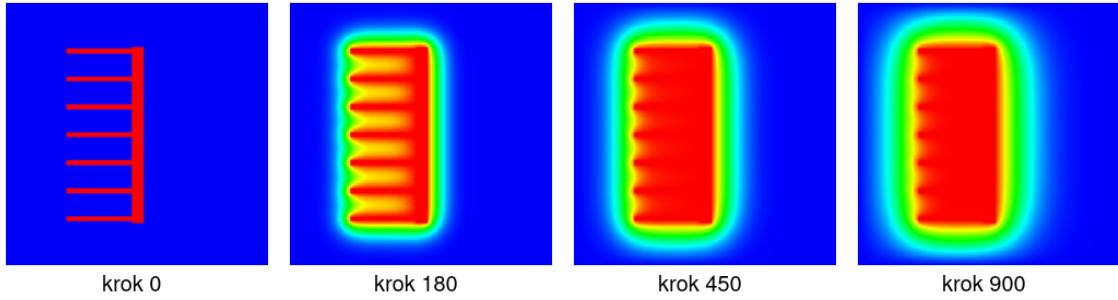
gdzie ∇^2 oznacza operator Laplace’a, natomiast $\alpha > 0$ to współczynnik dyfuzyjności cieplnej (ang. *thermal diffusivity*), wyrażony w jednostkach m^2/s . Wielkość α łączy przewodność cieplną ośrodka z jego pojemnością cieplną i gęstością; im większa jej wartość, tym szybciej lokalne różnice temperatury ulegają wygładzeniu. W zaimplementowanym programie przyjęto bezwymiarową wartość $\alpha = 0,25$ jako parametr konfiguracyjny.

2.1.2 Warunki początkowe i brzegowe

Rozwiązanie równania (2.1) jest jednoznaczne dopiero po zadaniu warunku początkowego oraz warunków brzegowych. W zaimplementowanym rozwiązaniu warunek początkowy przyjmuje jednorodne pole $u(\mathbf{x}, 0) = 1$ w całej dziedzinie, w którym umieszczono nagrzaną warstwę o grubości `hotThickness` płaszczyzn siatki przyle-

gających do ściany $y = 0$, o podwyższonej wartości $u = 2$. Warstwa ta modeluje gorącą ścianę, od której ciepło rozchodzi się w głąb sześcianu.

Na brzegu dziedziny przyjęto warunki Dirichleta o ustalonych wartościach: cała zewnętrzna warstwa komórek siatki nigdy nie jest aktualizowana i zachowuje swoje wartości początkowe przez cały czas symulacji. W praktyce oznacza to, że aktualizowane jest wyłącznie wnętrze dziedziny, a warstwa gorąca przy $y = 0$ pozostaje utrzymywana na poziomie $u = 2$, pełniąc rolę stałego źródła ciepła. Pozostałe ściany, utrzymywane na poziomie $u = 1$, działają jak zbiorniki ciepła o ustalonej temperaturze. Taki dobór warunków daje pole, które monotonnie ewoluuje i z czasem dąży do stanu ustalonego, z gradientem między gorącą a zimnymi ścianami. Rysunek 2.1 ilustruje jakościowo charakter tej ewolucji na dwuwymiarowym przykładzie: podtrzymywane źródło ciepła stopniowo traci ostre krawędzie, a ciepło rozchodzi się w otoczenie.



Rysunek 2.1: Ewolucja pola temperatury w dwuwymiarowej symulacji dyfuzji, wykonana dwuwymiarowym programem Chapel (dziedzina 160×160). Od lewej: początkowe, podtrzymywane źródło ciepła w kształcie grzebienia, a następnie stopniowe wygładzanie ostrych szczegółów i propagacja ciepła w otoczenie, aż do ukształtowania się gładkiego gradientu. Barwa odwzorowuje temperaturę: błękit oznacza zimno, czerwień gorąco. Ta sama zasada obowiązuje w przypadku trójwymiarowym rozważanym w pracy.

2.1.3 Dyskretyzacja: jawna metoda różnic skończonych

Do rozwiązywania równania (2.1) zastosowano jawną metodę różnic skończonych (ang. *Finite Difference Method*, FDM) z jawnym schematem Eulera w czasie i centralnymi różnicami drugiego rzędu w przestrzeni [12, 13]. Dziedzinę ciągłą zastępuje się regularną siatką kartezjańską o $n_x \times n_y \times n_z$ węzłach. Krok siatki w każdym z kierunków dobrany jest tak, aby węzły skrajne pokrywały się z brzegiem sześcianu jednostkowego:

$$\Delta x = \frac{1}{n_x - 1}, \quad \Delta y = \frac{1}{n_y - 1}, \quad \Delta z = \frac{1}{n_z - 1}. \quad (2.2)$$

Wartość pola w węźle (i, j, k) w kroku czasowym n oznacza się przez $u_{i,j,k}^n$. Do przybliżenia pierwszej pochodnej czasowej użyto ilorazu różnicowego przedniego, a drugich pochodnych przestrzennych standardowych trzypunktowych ilorazów centralnych, na przykład dla kierunku x :

$$\frac{\partial^2 u}{\partial x^2}(x_i, y_j, z_k) \approx \frac{u_{i-1,j,k}^n - 2u_{i,j,k}^n + u_{i+1,j,k}^n}{\Delta x^2}, \quad (2.3)$$

i analogicznie dla kierunków y oraz z . Podstawienie tych przybliżeń do (2.1) i rozwiązanie względem wartości w następnym kroku prowadzi do jawnej formuły aktualizacji.

Dla zwięzłości wprowadza się współczynniki wagowe zależne od osi, łączące dyfuzyjność, krok czasowy oraz krok siatki:

$$a_x = \frac{\alpha \Delta t}{\Delta x^2}, \quad a_y = \frac{\alpha \Delta t}{\Delta y^2}, \quad a_z = \frac{\alpha \Delta t}{\Delta z^2}. \quad (2.4)$$

Przy tych oznaczeniach regułę aktualizacji można zapisać jako siedmiopunktowy stencil (ang. *7-point stencil*):

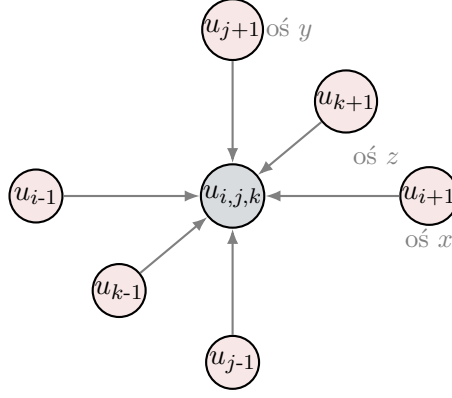
$$\begin{aligned} u_{i,j,k}^{n+1} = & u_{i,j,k}^n + a_x (u_{i-1,j,k}^n - 2u_{i,j,k}^n + u_{i+1,j,k}^n) \\ & + a_y (u_{i,j-1,k}^n - 2u_{i,j,k}^n + u_{i,j+1,k}^n) \\ & + a_z (u_{i,j,k-1}^n - 2u_{i,j,k}^n + u_{i,j,k+1}^n). \end{aligned} \quad (2.5)$$

Nowa wartość w każdym węźle zależy wyłącznie od wartości w tym węźle oraz w jego sześciu bezpośrednich sąsiadach z poprzedniego kroku czasowego: dwóch wzdłuż każdej z trzech osi. Ten wzorzec dostępu do pamięci, komórka centralna plus sześć sąsiadów, przedstawiono na rys. 2.2. Zależność od stanu wyłącznie z poprzedniego kroku jest cechą schematu jawnego i czyni każdy krok czasowy w pełni równoległym względem węzłów siatki.

2.1.4 Stabilność numeryczna i warunek CFL

Zaletą schematów jawnych jest prostota i niski koszt pojedynczego kroku: nie wymagają one rozwiązywania układów równań, w przeciwieństwie do schematów niejawnych. Ceną za tę prostotę jest jednak warunkowa stabilność: krok czasowy Δt nie może być dowolnie duży [12, 14]. Zbyt duży krok powoduje, że błędy zaokrągleń i składowe wysokoczęstotliwościowe rozwiązania numerycznego są wzmacniane w każdej iteracji zamiast tłumione, co prowadzi do gwałtownej, niefizycznej rozbieżności rozwiązania.

Analiza stabilności metodą von Neumanna [13, 12] dla schematu jawnego



Rysunek 2.2: Siedmiopunktowy stencil trójwymiarowy. Nowa wartość w komórce centralnej $u_{i,j,k}$, zaznaczonej na niebiesko, jest ważoną kombinacją jej wartości bieżącej oraz sześciu bezpośrednich sąsiadów zaznaczonych na czerwono, wzdłuż osi x , y i z , zgodnie z równaniem (2.5).

zastosowanego do równania dyfuzji w trzech wymiarach prowadzi do warunku Couranta–Friedrichsa–Lewy’ego (CFL). Dla liniowego schematu o stałych współczynnikach współczynnik wzmocnienia pojedynczego modu Fouriera wynosi $g = 1 - 4a_x \sin^2(\theta_x/2) - 4a_y \sin^2(\theta_y/2) - 4a_z \sin^2(\theta_z/2)$, a żądanie $|g| \leq 1$ dla najbardziej niekorzystnego modu o naprzemiennym znaku między sąsiednimi węzłami redukuje się do nierówności $a_x + a_y + a_z \leq \frac{1}{2}$: sumaryczna waga dyfuzyjna nie może przekraczać jednej drugiej, co wyznacza maksymalny dopuszczalny krok czasowy:

$$\Delta t_{\max} = \frac{1}{2\alpha \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} + \frac{1}{\Delta z^2} \right)}. \quad (2.6)$$

Zależność (2.6) ma praktyczne konsekwencje: dwukrotne zagęszczenie siatki wymusza czterokrotne zmniejszenie kroku czasowego, a więc czterokrotny wzrost liczby kroków potrzebnych do przebycia tego samego przedziału czasu fizycznego. Wraz z ośmiokrotnym wzrostem liczby węzłów siatki daje to duży wzrost całkowitego nakładu obliczeniowego, co jest jedną z przyczyn, dla których symulacje dużych siatek wymagają obliczeń rozproszonych.

W zaimplementowanym programie krok czasowy dobierany jest automatycznie na podstawie warunku (2.6), z zastosowaniem współczynnika bezpieczeństwa 0,98:

$$\Delta t = 0,98 \cdot \Delta t_{\max}. \quad (2.7)$$

Ten 2% margines (ang. *safety back-off*) przesuwą punkt pracy poniżej granicy stabilności, do $g = -0,96$ dla najbardziej niekorzystnego modu. Dzięki temu schemat pozostaje stabilny nawet przy błędach zaokrągleń arytmetyki zmiennoprzecinkowej i drobnych efektach numerycznych, które mogłyby zepchnąć krok dokładnie na gra-

nicy w obszar niestabilności. Wartość bliska jedności jest też korzystna wydajnościowo: maksymalizuje postęp w czasie fizycznym na jeden krok i zmniejsza liczbę wymian danych między węzłami, o których mowa w dalszej części rozdziału.

2.2 Zrównoleglenie obliczeń stencilowych

2.2.1 Dekompozycja dziedziny i komórki brzegowe

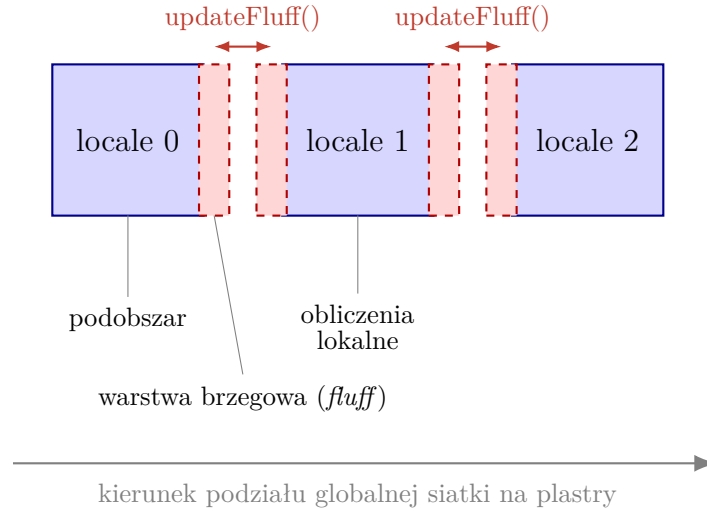
Obliczenia stencilowe należą do klasy problemów zrównoleglanych przez podział danych (ang. *data parallelism*). Ponieważ aktualizacja każdego węzła według (2.5) jest niezależna w obrębie jednego kroku czasowego, naturalną strategią jest dekompozycja dziedziny (ang. *domain decomposition*): globalna siatka zostaje podzielona na rozłączne podobszary, z których każdy przypisany jest do osobnej jednostki obliczeniowej, w rozważanym przypadku do osobnego węzła klastra. Każdy węzeł oblicza aktualizację wyłącznie dla przydzielonych mu komórek.

Problem pojawia się na granicach podobszarów. Aby zaktualizować komórki leżące tuż przy krawędzi swojego podobszaru, węzeł potrzebuje wartości sąsiadów znajdujących się już w podobszarze sąsiedniego węzła. Rozwiązaniem jest otoczenie każdego lokalnego podobszaru dodatkową warstwą komórek brzegowych (ang. *ghost cells* lub *halo*), które przechowują kopie wartości z krawędzi sąsiednich podobszarów. W terminologii Chapel warstwa ta nosi nazwę *fluff*; w zaimplementowanym programie zadeklarowano ją o grubości jednej komórki w każdym kierunku, $\text{fluff}=(1,1,1)$, co odpowiada zasięgowi siedmiopunktowego stencilu. Rysunek 2.3 przedstawia schematycznie podział dziedziny na podobszary wraz z warstwami brzegowymi.

2.2.2 Wymiana brzegów jako jedyna komunikacja

W każdym kroku czasowym warstwy brzegowe muszą zostać odświeżone, zanim rozpocznie się faza obliczeń, gdyż przechowywane w nich wartości pochodzą z poprzedniego kroku sąsiedniego węzła. Ta wymiana danych z najbliższymi sąsiadami (ang. *nearest-neighbour* lub *halo exchange*) jest jedynym rodzajem komunikacji między węzłami wymaganym przez schemat jawny. W zaimplementowanym programie realizuje ją pojedyncze wywołanie metody `updateFluff()` biblioteki rozkładów Chapel, wykonywane na początku każdego kroku, przed pętlą aktualizującą wnętrze dziedziny.

Prowadzi to do wyraźnego rozdzielenia każdego kroku czasowego na dwie fazy: fazę komunikacji, czyli odświeżenie warstw brzegowych, oraz fazę obliczeń, czyli wykonanie formuły (2.5) dla wszystkich komórek wnętrza. W programie fazy te są



Rysunek 2.3: Dekompozycja dziedziny w postaci plastrów (ang. *slab decomposition*) na trzy jednostki (*locale*). Globalna siatka jest cięta wzdłuż jednej osi, którą wskazuje strzałka na dole; każdy podobszar, zaznaczony na niebiesko, otoczony jest warstwą komórek brzegowych *fluff*, zaznaczoną na czerwono linią przerywaną, przechowującą kopie krawędzi sąsiadów. Przed każdym krokiem obliczeń wywołanie `updateFluff()` realizuje wymianę brzegów z najbliższymi sąsiadami.

oddzielnie mierzone stoperami `fluffTimer` i `computeTimer`, co pozwala rozdzielić czas spędzony na komunikacji od czasu obliczeń. Struktura pojedynczego kroku jest zwięźle widoczna w procedurze `stepOnce`, przedstawionej na listing 2.1: najpierw synchronizacja brzegów, potem równoległa aktualizacja wnętrza.

```

1 proc stepOnce(ref dst, ref src, ref fluffTimer, ref computeTimer) {
2   fluffTimer.start();
3   src.updateFluff();           // jedyna komunikacja: wymiana
   halo
4   fluffTimer.stop();
5   computeTimer.start();
6   forall (i,j,k) in interior do // faza obliczeń, w pełni ró
   wnoległa
7     dst[i,j,k] = src[i,j,k]
8       + ax * (src[i-1,j,k] - 2*src[i,j,k] + src[i+1,j,k])
9       + ay * (src[i,j-1,k] - 2*src[i,j,k] + src[i,j+1,k])
10      + az * (src[i,j,k-1] - 2*src[i,j,k] + src[i,j,k+1]);
11   computeTimer.stop();
12 }
```

Listing 2.1: Pojedynczy krok czasowy: wymiana brzegów, a następnie równoległa aktualizacja wnętrza dziedziny.

2.2.3 Intensywność arytmetyczna i ograniczenia wydajności

Obliczenia stencilowe mają bardzo niską *intensywność arytmetyczną* (ang. *arithmetic intensity*) [16], czyli iloraz liczby wykonanych operacji zmiennoprzecinkowych do liczby bajtów przeniesionych z pamięci. Aktualizacja jednego węzła według (2.5) wymaga zaledwie kilkunastu operacji zmiennoprzecinkowych, sześciu odejmowań, trzech mnożeń przez wagi i kilku dodawań, a jednocześnie odczytu siedmiu wartości i zapisu jednej, przy 8 B na liczbę w podwójnej precyzji daje to zaledwie ułamek operacji na każdy przeniesiony bajt. Co więcej, sąsiednie węzły współdzielą większość odczytów, lecz nawet przy idealnym wykorzystaniu pamięci podręcznej stosunek ten pozostaje niski.

W efekcie wydajność obliczeń stencilowych ogranicza nie moc procesora, lecz przepustowość transferu danych. W ujęciu modelu *roofline* [15] punkt pracy tego typu zadania leży daleko po lewej stronie wykresu, w obszarze ograniczonym nachyloną prostą przepustowości pamięci, a nie poziomą półką szczytowej wydajności obliczeniowej. Na pojedynczym węźle oznacza to ograniczenie przepustowością pamięci (ang. *memory-bandwidth bound*): dodawanie kolejnych rdzeni obliczeniowych przynosi coraz mniejsze korzyści, gdy magistrała pamięci ulega nasyceniu. W obliczeniach rozproszonych, gdzie warstwy brzegowe muszą przejść przez sieć, zadanie staje się ograniczone komunikacją (ang. *communication bound*): o całkowitym czasie decyduje przepustowość i opóźnienie połączeń sieciowych między węzłami, a nie szybkość samych obliczeń.

Zależność ta ma jednak korzystną stronę. Objętość obliczeń w podobszarze rośnie proporcjonalnie do jego objętości, czyli jak N^3 dla sześcianu o boku N , podczas gdy objętość danych brzegowych podlegających wymianie rośnie proporcjonalnie do pola powierzchni granic, czyli jak N^2 . Stosunek nakładu obliczeń do nakładu komunikacji jest zatem rzędu $N^3/N^2 = N$ i rośnie liniowo z rozmiarem krawędzi siatki. Większe sześciany mają zatem korzystniejszy stosunek obliczeń do komunikacji, bo koszt komunikacji zostaje zamortyzowany większą liczbą obliczeń przypadających na każdy przesłany bajt. Wynik ten sprawdzono empirycznie w rozdziale z wynikami.

2.2.4 Miary skalowalności

Do ilościowej oceny efektywności zrównoleglenia stosuje się standardowy aparat pojęciowy. *Przyspieszenie* (ang. *speedup*) przy użyciu p jednostek obliczeniowych definiuje się jako stosunek czasu wykonania sekwencyjnego do czasu wykonania równoległego:

$$S(p) = \frac{T(1)}{T(p)}, \quad (2.8)$$

a efektywność równoległą (ang. *parallel efficiency*) jako przyspieszenie znormalizowane liczbą jednostek:

$$E(p) = \frac{S(p)}{p} = \frac{T(1)}{pT(p)}. \quad (2.9)$$

Idealne, liniowe przyspieszenie odpowiada $S(p) = p$ oraz $E(p) = 1$; w praktyce narzuty komunikacji, synchronizacji i nierównomiernego obciążenia sprawiają, że $E(p) < 1$, przy czym efektywność zwykle maleje ze wzrostem p .

Wyróżnia się dwa rodzaje skalowania. *Skalowanie mocne* (ang. *strong scaling*) polega na ustaleniu całkowitego rozmiaru problemu i zwiększaniu liczby jednostek obliczeniowych; celem jest skrócenie czasu rozwiązania. Przy tym skalowaniu udział pracy przypadającej na jednostkę maleje, przez co narzuty komunikacyjne stają się względnie coraz istotniejsze, a granicę osiągalnego przyspieszenia opisuje prawo Amdahla. *Skalowanie słabe* (ang. *weak scaling*) polega na ustaleniu rozmiaru problemu przypadającego na jedną jednostkę i proporcjonalnym zwiększaniu zarówno liczby jednostek, jak i całkowitego rozmiaru problemu; celem jest rozwiązywanie coraz większych zadań w stałym czasie.

Fundamentalne ograniczenie skalowania mocnego formułuje *prawo Amdahla* [17]. Jeśli f oznacza sekwencyjną część obliczeń, której z natury nie da się zrównoleglić, to maksymalne osiągalne przyspieszenie przy p jednostkach wynosi

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}}, \quad \lim_{p \rightarrow \infty} S(p) = \frac{1}{f}. \quad (2.10)$$

Nawet niewielki udział części sekwencyjnej narzuca twardą górną granicę przyspieszenia, niezależnie od liczby dostępnych jednostek. W kontekście rozważanego programu rolę nieusuwalnego narzutu pełni głównie komunikacja, wymiana warstw brzegowych oraz synchronizacja między krokami, co, w połączeniu z ograniczeniem przepustowości pamięci i sieci omówionym wcześniej, wyznacza granice skalowalności, które badano w rozdziale z wynikami.

2.3 Chapel, PGAS i GASNet

2.3.1 Język Chapel i model PGAS

Implementację programu oparto na języku Chapel, języku programowania równoległego rozwijanym z naciskiem na wygodę programisty przy zachowaniu wysokiej wydajności na systemach wieloprocessorowych i rozproszonych [1, 2]. Chapel realizuje model *PGAS* (ang. *Partitioned Global Address Space*, partycjonowana globalna przestrzeń adresowa) [8, 9]. Idea PGAS polega na tym, że pamięć wszystkich wę-

złów tworzy jedną logicznie wspólną przestrzeń adresową, która jest jednak podzielona na partycje o jawnie określonym powinowactwie do węzłów. Programista operuje na danych za pomocą jednolitej, globalnej perspektywy (ang. *global-view*), tak jakby były one lokalne, natomiast środowisko uruchomieniowe automatycznie tłumaczy dostępy do danych zdalnych na operacje komunikacji sieciowej. W ten sposób PGAS łączy wygodę programowania w pamięci wspólnej, typową dla modelu pamięci współdzielonej, z kontrolą rozmieszczenia danych, którą daje model przekazywania komunikatów.

Chapel nie jest jedynym językiem tej grupy. Poza nim model PGAS realizują m.in. UPC (ang. *Unified Parallel C*), rozszerzenie języka C o globalne wskaźniki, Co-Array Fortran z notacją współobrazów, który wszedł do standardu Fortran 2008, a także języki projektowane od podstaw dla PGAS: X10 (IBM) i Fortress (Sun Microsystems). Rozwiązania te różnią się głównie składnią, modelem zadań i sposobem wyrażania lokalności; wspólne jest im oddanie programiście globalnego widoku danych przy jawnym oznaczaniu powinowactwa do węzłów [8, 9].

Centralnym pojęciem w Chapel jest *locale*, abstrakcja jednostki systemu dysponującej własną pamięcią oraz zdolnością do wykonywania obliczeń. W konfiguracji wielowęzłowej stosowanej w niniejszej pracy każdemu węzłowi fizycznemu klastra odpowiada dokładnie jeden locale, w odwzorowaniu *jeden-do-jednego*. Wbudowana tablica `Locales` udostępnia wszystkie dostępne jednostki, a ich liczbę podaje stała `numLocales`. Dostęp do danych i przenoszenie obliczeń pomiędzy jednostkami wyraża się wprost, za pomocą konstrukcji `on`.

2.3.2 Iteratory równoległe i tablice globalne

Chapel oferuje dwa uzupełniające się rodzaje równoległości. Równoległość danych wyraża pętla `forall`, która zleca wykonanie ciała iteracji dla wszystkich elementów dziedziny współbieżnie, pozostawiając środowisku uruchomieniowemu podział pracy na zadania i przypisanie ich do wątków. W programie to właśnie pętla `forall` nad dziedziną wnętrza (listing 2.1) realizuje fazę obliczeń stencila. Gdy pętla iteruje po dziedzinie rozproszonej, jej iteracje wykonują się na tych węzłach, do których przypisane są odpowiednie elementy, co zapewnia lokalność obliczeń bez jawnego zarządzania nią przez programistę.

Równoległość zadaniową między węzłami wyraża konstrukcja `coforall ... on loc`, która tworzy osobne zadanie dla każdego locale i umieszcza jego wykonanie na odpowiednim węźle. W programie wzorec ten wykorzystywany jest między innymi do równoległego zapisu klatek wynikowych, każdy węzeł niezależnie zapisuje swój lokalny podobszar. Listing 2.2 ilustruje ten idiom.

```

1 coforall loc in Locales do on loc {
2     const ld = field.localSubdomain();    // lokalny fragment tablicy
        globalnej
3     var block: [ld] real = field[ld];      // kopia danych bez
        komunikacji zdalnej
4     // ... zapis / obliczenia lokalne na fragmencie ...
5 }

```

Listing 2.2: Równoległość zadaniowa między węzłami: każdy locale wykonuje operację na swoim lokalnym podobszarze danych.

Tablice w Chapel definiowane są nad *dziedzinami* (ang. *domain*), obiektami opisującymi zbiór indeksów. Aby tablica została rozproszona pomiędzy węzły, jej dziedzinę wyposaża się w *odzworowanie dziedziny* (ang. *domain map*, dystrybucję). Najprostszą dystrybucją jest `Block`, dzieląca zakres indeksów na w przybliżeniu równe, przylegające bloki i przypisująca je kolejnym węzłom. Na potrzeby obliczeń stencilowych Chapel udostępnia wyspecjalizowaną dystrybucję `Stencil` z modułu `StencilDist`, która poza podziałem danych utrzymuje wokół każdego lokalnego bloku warstwę komórek brzegowych *fluff*, przechowującą buforowane kopie krawędzi sąsiadów. Warstwę tę odświeża się jawnym wywołaniem `updateFluff()`, co realizuje opisaną wcześniej wymianę *halo* [4]. W programie dziedzinę tworzy się wywołaniem `stencilDist.createDomain(..., fluff=(1,1,1))`, zapewniając brzeg o grubości jednej komórki, dokładnie odpowiadający zasięgowi stencilu (2.5).

2.3.3 Konstrukcje równoległości i synchronizacja

Oprócz pętli `forall` i konstrukcji `coforall...on` Chapel udostępnia pomocnicze mechanizmy równoległości i synchronizacji, które pozwalają wyrażać wzorce nie dające się zapisać samym podziałem danych.

Równoległość na poziomie instrukcji realizują `begin` i `cobegin`. Instrukcja `begin` uruchamia pojedyncze zadanie w tle i nie blokuje dalszego wykonywania bieżącego wątku, a `cobegin` wykonuje każde z podanych zdań równoległe i czeka na zakończenie ich wszystkich. Pozwala to na zagnieżdżanie zadań i tworzenie struktur, w których kilka operacji przeplata się w czasie.

```

1 cobegin {
2     writeTask();    // zadanie A
3     readTask();     // zadanie B, wykonuje się równoległe z A
4 }
5 // oba zadania kończą się zanim sterowanie przejdzie dalej

```

Listing 2.3: Równoległe uruchamianie zadań za pomocą `cobegin`.

Konstrukcja `on` przenosi wykonanie instrukcji na wskazany locale; wewnątrz bloku słowo kluczowe `here` oznacza bieżący locale, co pozwala odczytać jego właściwości bez jawnego odwoływania się do elementów tablicy `Locales`.

```
1 on Locales[1] do
2   writeln("dzialam na: ", here.id, " z ", here.maxTaskPar, "
   watkami");
```

Listing 2.4: Przeniesienie wykonania na konkretny locale z użyciem `here`.

Do synchronizacji producenta z konsumentem służą zmienne `single` (tak zwane zmienne synchronizujące): zapis do takiej zmiennej blokuje, dopóki nie zostanie odczytana, a odczyt — dopóki nie zostanie zapisana. Dają one semantykę jednorazowego przekazania wartości między zadaniami bez jawnych zamków.

```
1 var s: single int;           // zmienna synchronizujaca
2 begin { s = computeResult(); } // producent
3 const v = s;                 // konsument czeka na wartosc
```

Listing 2.5: Zmienna synchronizująca jako kanał jednorazowy między zadaniami.

Dla bezpiecznej manipulacji stanem współdzielonym Chapel udostępnia typy `atomic`, które gwarantują niepodzielność operacji odczytu, zapisu i dodawania. Typowe zastosowanie to liczniki globalne, sumy częściowe i sekwencjonowanie dostępu do zasobów, bez ryzyka wyścigu.

```
1 var counter: atomic int;
2 counter.add(1);           // atomowe dodawanie
3 const n = counter.read(); // atomowy odczyt
```

Listing 2.6: Operacje atomowe na liczniku współdzielonym.

Wreszcie, dla typowych operacji redukcyjnych Chapel oferuje wyrażenia redukcji, które pozwalają zredukować tablicę lub pętlę do pojedynczej wartości przy użyciu operatorów takich jak `+`, `min` czy `max`, przy czym samo dzielenie pracy i akumulacja wyniku pozostają w rękach środowiska uruchomieniowego.

```
1 var s = + reduce A; // suma wszystkich elementow tablicy A
```

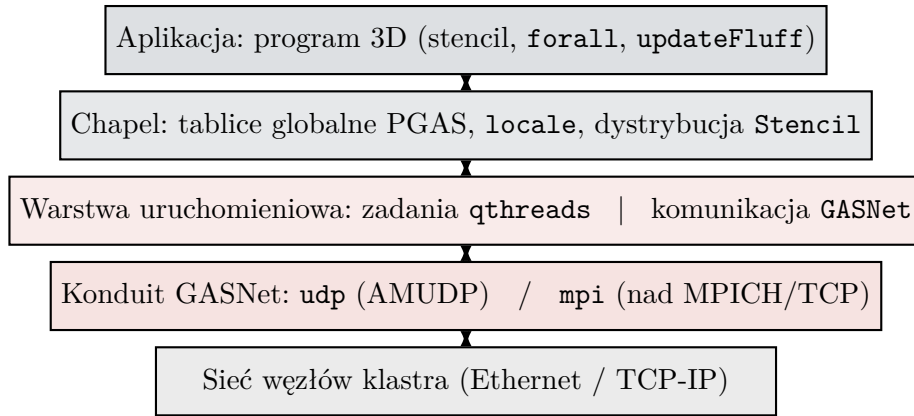
Listing 2.7: Redukcja sumy z pętli równoległej.

2.3.4 Warstwa zadań `qthreads` oraz warstwa komunikacji GASNet

Wykonywanie zadań w obrębie pojedynczego locale obsługuje scheduler zadań. W zastosowanej konfiguracji jest to biblioteka `qthreads` (`CHPL_TASKS=qthreads`), lekki

system wątków użytkownika efektywnie obsługujący dużą liczbę fine-grained zadań generowanych przez pętle `forall` [5]. Liczbę wątków sprzętowych dostępnych na danym węźle udostępnia własność `here.maxTaskPar`; wyznacza ona stopień równoległości fazy obliczeń w obrębie węzła i jest raportowana przez program przy starcie.

Za komunikację między węzłami odpowiada z kolei warstwa *GASNet*, przenośna biblioteka uruchomieniowa dostarczająca prymitywów komunikacji jednostronnej (ang. *one-sided*) oraz komunikatów aktywnych, na których Chapel opiera realizację dostępu do pamięci zdalnej i operacji `on` [6, 7]. Uaktywnia się ją ustawieniem `CHPL_COMM=gasnet`. W zastosowanej konfiguracji przyjęto tryb segmentu `CHPL_GASNET_SEGMENT=everything`, w którym cała pamięć procesu jest udostępniana dla komunikacji zdalnej, co upraszcza rozmieszczanie tablic rozproszonych. Warstwową strukturę całego stosu, od kodu aplikacji, przez model globalnej tablicy PGAS, aż po sprzęt sieciowy, przedstawia rys. 2.4.



Rysunek 2.4: Warstwowa architektura wykonania. Kod aplikacji operuje na globalnych tablicach PGAS języka Chapel; ich dostępy zdalne i wymiana warstw brzegowych są realizowane przez warstwę uruchomieniową, w której zadania `qthreads` wykonują się lokalnie, a komunikacja *GASNet* zachodzi między węzłami; warstwa ta korzysta z kolei z wybranego konduitu odwzorowującego operacje na fizyczną sieć klastra.

2.3.5 Konduity *GASNet*

GASNet abstrahuje sprzęt sieciowy poprzez wymienne *konduity* (ang. *conduits*), moduły implementujące prymitywy komunikacji na konkretnej technologii transportu. W ramach niniejszej pracy porównywano dwa konduity dostępne na klastrze zbudowanym z węzłów połączonych zwykłą siecią Ethernet. Konduit `udp`, oparty na warstwie AMUDP, realizuje komunikaty aktywne bezpośrednio nad datagramami UDP i jest uruchamiany za pomocą programu rozruchowego `amudprun`. Jego zaletą jest prosta konfiguracja, a wadą podatność na przeciążenie przy intensywnej, fine-grained wymianie danych. Konduit `mpi` przenosi komunikację *GASNet* na istniejącą

implementację MPI, w zastosowanej konfiguracji opartej na MPICH nad TCP, i jest uruchamiany programem `gasnetrun_mpi`; zapewnia on stabilniejszy i wydajniejszy transport dla podtrzymywanych, regularnych wzorców komunikacji, jakie generuje cykliczna wymiana warstw brzegowych w opisywanym programie.

Wybór konduitu ma istotne znaczenie dla rozproszonej symulacji. Jak wykazano w podrozdz. 2.2, przy skalowaniu na wiele węzłów zadanie jest ograniczone komunikacją, a jedyną komunikacją w kroku jest wymiana *halo* realizowana przez `updateFluff()`. O osiąganym skalowalności decydują w praktyce właściwości warstwy komunikacyjnej: przepustowość, opóźnienie i odporność na przeciążenie. Dlatego dobór konduitu jest jednym z głównych pytań badawczych w tej pracy.

Rozdział 3

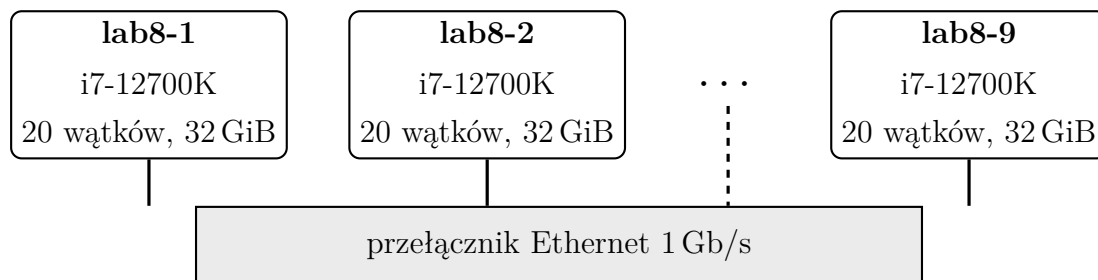
Klaster i implementacja

3.1 Architektura klastra i środowisko

3.1.1 Sprzęt i topologia

Docelowym środowiskiem uruchomieniowym jest rzeczywisty klaster *Pionier* w pracowni komputerowej Wydziału Matematyki i Informatyki UMK, złożony z dziewięciu jednorodnych węzłów fizycznych (*Pionier-lab8-1* ... *Pionier-lab8-9*) połączonych przełącznikiem Ethernet o przepustowości 1 Gb/s. Wszystkie węzły są identyczne: każdy wyposażony jest w procesor Intel Core i7-12700K (12 rdzeni, 20 wątków sprzętowych), 32 GiB pamięci RAM DDR4-3200 oraz kartę sieciową Intel I219-V (sterownik `e1000e`, 1 Gb/s) i pracuje pod systemem Ubuntu (jądro `6.8.0-111-generic`). Węzły mają adresy z podsieci `192.168.128.0/24`, a dostęp do nich odbywa się bezpośrednim poleceniem `ssh <ip>`. Jednorodność środowiska, czyli ta sama architektura procesora, dystrybucja i wersja bibliotek na każdym węźle, upraszcza dystrybucję plików wykonywalnych: obraz zbudowany na jednym węźle działa bez zmian na pozostałych. W eksperymentach skalowania każdemu węzłowi fizycznemu odpowiadał jeden lokal (*locale*), co pozwoliło zbadać skalowanie w funkcji liczby węzłów aż do dziewięciu.

Topologię środowiska przedstawiono na rys. 3.1. Wszystkie węzły dzielą jedno łącze o przepustowości 1 Gb/s, które jest wąskim gardłem komunikacji rozproszonej, bo każda wymiana komórek brzegowych między sąsiednimi fragmentami dziedziny przechodzi przez ten sam przełącznik.



Rysunek 3.1: Topologia jednorodnego, dziewięciowęzłowego klastra Pionier. Wszystkie identyczne węzły (*Pionier-lab8-1* ... *Pionier-lab8-9*) połączone są jednym przełącznikiem 1 Gb/s, który stanowi współdzielone i zarazem najwęższe łącze komunikacji rozproszonej. Środowisko budowane jest raz na dowolnym węźle, a pliki wykonywalne rozprowadzane na pozostałe.

3.1.2 Konfiguracja kompilatora Chapel

Kompilator Chapel w wersji 2,9,0 zbudowano ze źródeł na jednym węźle [3]. Kluczowe parametry środowiska są „zapieczone” w pliku `chplconfig`, który skrypt `distribute-chapel.sh` zapisuje jeszcze przed wywołaniem `make`. Dzięki temu każdy węzeł otrzymuje pliki wykonywalne zbudowane z identyczną konfiguracją. Zawartość istotnej części pliku konfiguracyjnego przedstawia listing 3.1.

```

1 CHPL_COMM=gasnet
2 CHPL_COMM_SUBSTRATE=$CONDUIT          # udp (domyslnie) albo mpi
3 CHPL_LLVM=none                        # backend C, nie LLVM
4 CHPL_TARGET_CPU=$TARGET_CPU           # native

```

Listing 3.1: Fragment pliku `chplconfig` generowanego przed budową kompilatora.

Warstwą komunikacyjną jest GASNet (`CHPL_COMM=gasnet`), co umożliwia pracę wielolokalową. Kompilacja odbywa się z użyciem backendu C (`CHPL_LLVM=none`) zamiast LLVM. Ustawienie `CHPL_TARGET_CPU=native` pozwala na generowanie kodu wykorzystującego pełen zestaw instrukcji procesora hosta; jest to bezpieczne wyłącznie dlatego, że produkcyjny klaster jest jednorodny pod względem architektury procesorów. Z konfiguracji wynikają dalsze, domyślne wartości środowiska: `CHPL_TASKS=qthreads`, `CHPL_GASNET_SEGMENT=everything`, `CHPL_LOCALE_MODEL=flat` oraz `CHPL_TARGET_MEM=jemalloc`. Launcher zależy od wybranego kanału komunikacyjnego i przyjmuje wartość `amudprun` dla kanału `udp` albo `gasnetrun_mpi` dla kanału `mpi`.

3.1.3 Dwa kanały komunikacyjne GASNet

Warstwę GASNet można zbudować na dwóch różnych kanałach transportowych (*conduit*), wybieranych parametrem `--conduit` [7]. Kanał **udp** realizuje protokół

AMUDP nad UDP; nie ma dodatkowych zależności i jest szybki w czystej sieci lokalnej, jednak przy dużej skali załamuje się z błędem `ECONGESTION` w sytuacji wielu-do-jednego (*incast*) oraz przy utracie pakietów [18]. Kanał `mpi` realizuje GASNet `mpi-conduit` nad biblioteką MPICH pracującą w trybie `ch3:sock`, czyli zwykłym TCP; mechanizm kontroli przepływu TCP przetrwa utratę pakietów i zjawisko *incast* [10, 11]. Ze względu na to, że wybór kanału zapisany jest w pliku `chplconfig`, dla każdego kanału utrzymywane jest osobne, w pełni skompilowane drzewo Chapel (ścieżki `.../chapel` oraz `.../chapel-mpi`).

Biblioteka MPICH w wersji 4,2,3 została zbudowana ze źródeł skryptem `build-mpi.sh` do bezuprawnieniowego prefiksu, z linią konfiguracji przedstawioną w listing 3.2. Budowa ze źródeł zapewnia na każdym węźle *identyczną* wersję i konfigurację MPI, w szczególności urządzenie `ch3:sock`, czyli zwykły TCP, którego mechanizm kontroli przepływu przetrwa *incast*, czego nie gwarantują pakiety dystrybucyjne, dostarczające zwykle OpenMPI w domyślnej konfiguracji. Jedno drzewo zbudowane raz jest kopiowane pod tę samą bezwzględną ścieżkę na każdym węźle, co gwarantuje zgodność ABI.

```
1 ./configure --prefix=$PREFIX --disable-fortran \
2           --with-device=ch3:sock --disable-static
```

Listing 3.2: Linia konfiguracji budowy MPICH ze źródeł.

3.1.4 Uruchamianie i samoopisujące się logi

Dla każdej binarki skrypt `compile-and-distribute.sh` (funkcja `gen_run_script`) generuje osobny skrypt uruchomieniowy. Ustawia on zmienne `CHPL_HOME` oraz `PATH`, domyślnie przyjmuje liczbę wątków na lokal równą `$(nproc)`, a przebieg rejestruje do logu o nazwie zawierającej znacznik czasu (`logs/<bin>-YYYYmdd-HHMMSS.log`). Zasadniczy fragment logiki uruchomieniowej pokazano w listing 3.3.

```
1 set -o pipefail
2 {
3     echo "[cfg] threadsPerLocale(requested)=
4         $CHPL_RT_NUM_THREADS_PER_LOCALE numLocales=${NUM_LOCALES}"
5     ./${bin} -nl ${NUM_LOCALES} "$@" 2>&1
6 } | tee "$LOG"
```

Listing 3.3: Blok uruchomieniowo-rejestrujący ze skryptu wrappera.

Log jest samoopisujący dzięki dwóm liniom `[cfg]`. Pierwsza, pochodząca ze skryptu, ma postać `[cfg] threadsPerLocale(requested)=N numLocales=M` i podaje wartości *żądane*. Druga, pochodząca z programu, ma postać `[cfg] locale`

<id> maxTaskPar=<n> i podaje *efektywną* liczbę wątków na lokal po ograniczeniu jej przez qthreads do możliwości sprzętu [5]. Rozróżnienie to jest istotne przy analizie wydajności, ponieważ w przypadku nadsubskrypcji (*oversubscription*) liczba żądana może przewyższać liczbę faktycznie dostępnych rdzeni. Konfiguracja launchera zależy od kanału: dla mpi ustawiane jest `MPIRUN_CMD="mpirun -n %N -hosts <hosts> <install>/mpi-iface-wrap.sh %C"`, natomiast dla udp, zmienne `GASNET_SSH_SERVERS` oraz `GASNET_MASTERIP`.

Do celowego wywoływania obciążenia sieci służy harness `run-nl.sh`. Przeplata on listę serwerów SSH tak, że kolejne lokale trafiają na różne węzły fizyczne, przez co sąsiadujące fragmenty dziedziny w rozkładzie `StencilDist` znajdują się na różnych węzłach. W konsekwencji *każda* operacja pobrania halo przekracza wspólne łącze 1 Gb/s, tworząc ruch wielu-do-jednego (incast), co uwiadamia błąd `ECONGESTION` kanału udp.

Ostatnim elementem środowiska jest skrypt `mpi-iface-wrap.sh` odpowiedzialny za przypięcie interfejsu sieciowego per ranga na węzłach wielosieciowych. Wybiera on lokalny adres IPv4 z właściwej podsieci klastra (192.168.128.0/24), pomijając interfejsy wirtualne, czyli mostki `docker0` i `virbr0`, podsieci 172.16/12 oraz adresy libvirt 192.168.122.*, eksportuje `MPICH_INTERFACE_HOSTNAME`, po czym uruchamia program. Bez tego kroku transport `ch3:sock` może rozgłosić nieroutowalny adres interfejsu, a wywołanie `MPI_Init` kończy się błędem „No route to host”.

3.1.5 Procedura wdrożenia krok po kroku

Elementy opisane w poprzednich podrozdziałach tworzą procedurę wdrożenia, od zbudowania środowiska aż po uruchomienie przebiegu. Choć węzły są jednorodne, procedura ta wykracza poza zwykłą kompilację lokalną, ponieważ kolejność kroków oraz miejsce ich wykonania są istotne dla poprawności i wymagają spójnych ścieżek oraz identycznych drzew Chapel i MPICH na wszystkich węzłach. Całość podzielono na cztery etapy.

W *pierwszym etapie* na dowolnie wybranym węźle budowane jest środowisko wykonawcze: kompilator Chapel w wersji 2,9,0 (skryptem `distribute-chapel.sh`) oraz biblioteka MPICH w wersji 4,2,3 w trybie `ch3:sock` (skryptem `build-mpi.sh`). Ponieważ klaster jest jednorodny, wybór węzła budującego jest dowolny, pliki wykonywalne zbudowane na jednym węźle działają na wszystkich pozostałych.

W *drugim etapie* skompilowane drzewo Chapel jest pakowane do archiwum i rozsyłane pod *identyczną* ścieżkę bezwzględną na każdym węźle (listing 3.4). Identyfikacja tej ścieżki jest drugim niezmiennikiem całej procedury: warstwa uruchomieniowa Chapel jest kompilowana ze ścieżką `rpath` wskazującą wprost do drzewa `CHPL_HOME`, więc jakakolwiek rozbieżność lokalizacji uniemożliwia dynamiczne odna-

leżenie bibliotek współdzielonych. Analogicznie, jedno drzewo MPICH zbudowane raz jest strumieniowane pod tę samą ścieżkę na wszystkich węzłach, co gwarantuje zgodność ABI wymaganą przez mpi-conduit.

```
1 tar czf chapel-2.9.0-built.tar.gz -C "$CHPL_HOME/.." chapel-2.9.0
2 for h in $HOSTS; do
3     scp chapel-2.9.0-built.tar.gz "$h:$DEST/"
4     ssh "$h" "tar xzf $DEST/chapel-2.9.0-built.tar.gz -C $DEST"
5 done
```

Listing 3.4: Spakowanie i rozesłanie zbudowanego drzewa pod identyczną ścieżkę.

W *trzecim etapie* kompilowany jest sam program i rozprowadzany na węzły skryptem `compile-and-distribute.sh`. Dla każdego programu kopiowane są *zarówno* plik launchera `<bin>`, jak i właściwy obraz wykonywalny `<bin>_real`; oba są niezbędne na każdym węźle do uruchomienia wielolokalowego. Skrypt generuje przy tym dedykowany launcher (por. listing 3.3), który ustawia `CHPL_HOME` i `PATH` zgodnie z rozesłanym drzewem.

W *czwartym etapie* przebieg uruchamiany jest przez wygenerowany launcher, który wywołuje właściwy program startowy GASNet (`gasnetrun_mpi` dla kanału mpi albo `amudprun` dla kanału udp) i rozdziela lokale na węzły. Dwa niezmienniki z poprzednich etapów, *identyczna ścieżka* `CHPL_HOME` (`rpath`) oraz *identyczne drzewa Chapel i MPICH* pod tą samą ścieżką na wszystkich węzłach, są warunkami koniecznymi powodzenia tego etapu. Naruszenie któregośkolwiek objawia się dopiero przy ładowaniu bibliotek lub starcie procesów zdalnych, czyli już po udanej kompilacji.

3.1.6 Uruchamianie eksperymentu

Procedurę pomiarową obsługują trzy skrypty generowane na węźle głównym. Skrypt `compile-and-distribute.sh` kompiluje program z wybranego conduitu, rozprowadza pliki wykonywalne na węzły z `hostfile'u` i tworzy na miejscu dwa dalsze skrypty: uruchomieniowy `run-heat3d.sh` oraz agregujący `aggregate-heat3d.sh`. Sam skrypt nie buduje kompilatora Chapel ani biblioteki MPICH: zakłada, że ich drzewa zostały uprzednio zbudowane i leżą pod tą samą ścieżką na każdym węźle (tworzą je skrypty `distribute-chapel.sh` oraz `build-mpi.sh`). Wynikowe pliki wykonywalne mają wbudowany `rpath` wskazujący na to drzewo, dlatego musi ono istnieć pod identyczną ścieżką na każdym węźle. Ten pierwszy wywołuje właściwy program z ustawionym środowiskiem Chapel i GASNet, przekazując mu swoje argumenty jako parametry `config` programu (patrz tab. 3.2); liczbę lokacji zapisuje w sobie na etapie generowania, zaś liczbę wątków na lokal ustala zmienna `CHPL_RT_NUM_THREADS_PER_LOCALE`. Skrypt agregujący zbiera rozproszone zrzuty

klatek na węzeł główny i uruchamia post-procesor `aggregate3d.chpl`, opcjonalnie renderując film.

Kluczowe flagi skryptu `compile-and-distribute.sh` zestawiono w tab. 3.1. Pełną listę podaje opcja `--help`.

Tabela 3.1: Wybrane flagi skryptu `compile-and-distribute.sh`.

Flaga	Wartość myślna	do- Znaczenie
<code>-f, --hostfile</code>	— (wymagane)	plik z adresami węzłów, master jako pierwszy
<code>-c, --conduit</code>	<code>udp</code>	kanał transportu: <code>udp</code> lub <code>mpi</code>
<code>-d, --dir</code>	<code>/home/<user></code>	zdalny katalog instalacji
<code>-u, --user</code>	<code>chapel</code>	użytkownik SSH
<code>-p, --port</code>	<code>22</code>	port SSH
<code>-V, --chapel-version</code>	<code>2.9.0</code>	wersja Chapela w ścieżce drzewa
<code>-m, --mpi-dir</code>	<code><dir>/mpi</code>	prefiks MPICH (kanał <code>mpi</code>)
<code>-o, --output</code>	<code>heat3d</code>	nazwa pliku wykonywalnego programu
<code>-s, --skip-compile</code>	wyłączony	pomiń kompilację, użyj istniejących plików wykonywalnych

Typowy przebieg pomiarowy składa się z trzech poleceń wykonywanych na węźle głównym. Najpierw budowa i dystrybucja, z kanałem `mpi` i wskazanym drzewem Chapel:

```
1 ./compile-and-distribute.sh -f hosts.txt -c mpi -d /home/chapel/
  workspace
```

Listing 3.5: Budowa i rozesłanie plików wykonywalnych na klaster.

Następnie uruchomienie programu na zadanej liczbie węzłów, z parametrami dziedziny i liczby kroków przekazanymi jako flagi `config`, oraz z żadaną liczbą wątków na lokal:

```
1 CHPL_RT_NUM_THREADS_PER_LOCALE=4 ./run-heat3d.sh --nx=1000 --
  numSteps=100
```

Listing 3.6: Uruchomienie przebiegu pomiarowego.

Na koniec, po serii przebiegów, scalenie rozproszonych zrzutów i opcjonalny render filmu:

```
1 ./aggregate-heat3d.sh --render=true
```

Listing 3.7: Agregacja rozproszonych fragmentów i wizualizacja.

3.2 Struktura programu i krok obliczeniowy

3.2.1 Dziedzina rozproszona z otoczką

Podstawą implementacji jest trójwymiarowa dziedzina rozproszona między lokale za pomocą rozkładu `StencilDist` [4]. Rozkład ten różni się od zwykłego rozkładu blokowego tym, że wokół bloku należącego do każdego lokala rezerwuje dodatkową warstwę komórek, tak zwaną otoczkę (*fluff*), przechowującą kopie komórek brzegowych sąsiadów. Szerokość otoczki dobrano do stosowanego siedmiopunktowego stencila, wynosi ona jedną komórkę w każdym kierunku. Definicję dziedziny oraz jej wnętrza przedstawia listing 3.8.

```
1 const domain3D = stencilDist.createDomain({0..<nx,0..<ny,0..<nz},  
2                                           fluff=(1,1,1));  
3 const interior = domain3D.expand(-1);
```

Listing 3.8: Definicja dziedziny rozproszonej z jednokomórkową otoczką oraz jej wnętrza.

Wnętrze `interior` powstaje przez zmniejszenie dziedziny o jedną komórkę w każdym kierunku (`expand(-1)`); obejmuje ono wyłącznie te komórki, dla których siedmiopunktowy stencil ma komplet sąsiadów wewnątrz dziedziny globalnej. Komórki brzegowe dziedziny nie są aktualizowane i zachowują wartości początkowe, pełniąc rolę warunków brzegowych typu Dirichleta.

3.2.2 Pojedynczy krok obliczeniowy

Pojedynczy krok czasowy realizuje procedura `stepOnce`, przedstawiona w listing 3.9. Składa się ona z dwóch wyraźnie rozdzielonych i osobno mierzonych faz: wymiany halo oraz właściwego obliczenia stencila. Najpierw wywoływana jest metoda `src.updateFluff()`, która wykonuje wymianę halo, następnie równoległa pętla `forall` po wnętrzu dziedziny aktualizuje pole zgodnie ze schematem różnic skończonych.

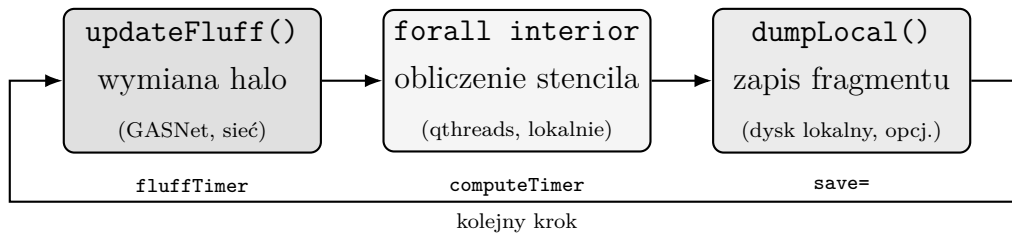
```

1 proc stepOnce(ref dst, ref src, ref fluffTimer, ref computeTimer) {
2   fluffTimer.start(); src.updateFluff(); fluffTimer.stop();
3   computeTimer.start();
4   forall (i,j,k) in interior do
5     dst[i,j,k] = src[i,j,k]
6                 + ax*(src[i-1,j,k]-2*src[i,j,k]+src[i+1,j,k])
7                 + ay*(src[i,j-1,k]-2*src[i,j,k]+src[i,j+1,k])
8                 + az*(src[i,j,k-1]-2*src[i,j,k]+src[i,j,k+1]);
9   computeTimer.stop();
10 }

```

Listing 3.9: Procedura pojedynczego kroku: wymiana halo, a następnie obliczenie stencila.

Metoda `updateFluff()` kopiuje komórki brzegowe każdego lokala do otoczek sąsiadów, przesyłając dane poprzez GASNet; jest to *jedyna* komunikacja odbywająca się w ramach kroku. Na więcej niż jednym lokalu wywołanie to jest *obowiązkowe* dla poprawności: pominięcie go powoduje odczyt nieaktualnych lub niezainicjowanych komórek otoczki, przez co obliczenie przebiega szybciej, lecz daje wynik *cicho błędny*. Dlatego wymiana halo jest jednocześnie gwarancją poprawności i głównym wąskim gardłem obliczeń rozproszonych, bo przy każdym kroku obciąża wspólne łącze 1 Gb/s. Przebieg pojedynczego kroku wraz z fazą zapisu ilustruje rys. 3.2.



Rysunek 3.2: Potok pojedynczego kroku czasowego programu. Faza komunikacyjna (`updateFluff`) poprzedza lokalne obliczenie stencila, po którym następuje opcjonalny zapis fragmentu na dysk lokalny. Każda faza mierzona jest osobnym stoperem.

3.2.3 Pomiar czasu i rejestrowanie

Program rozdziela pomiar czasu na wyraźnie zdefiniowane obszary. Stoper `initTimer` obejmuje fazę przygotowania (alokacja tablic, inicjalizacja gorącej warstwy oraz obliczenie kroku czasowego Δt i współczynników), której wynik raportowany jest jako *Initialization time*. Stoper `computeTimer` obejmuje całą pętlę czasową i raportowany jest jako *Execution time*. Dla każdego kroku wypisywana jest linia `step <n> updateFluff=<s> s compute=<s> s save=<s> s`. Ponadto rejestrator `commLog` wypisuje jednorazowo linię `[comm] <bytes> B/step across`

<N> locale(s) oraz przyrostowe liczby operacji put/get/on/amo na każdy krok, co pozwala powiązać obserwowany czas z faktyczną objętością komunikacji.

Efektywną liczbę wątków na lokal wypisuje serialna pętla `for` po lokalach, przez co linie są uporządkowane według identyfikatora lokala:

```
1 for loc in Locales do on loc do
2   writeln("[cfg] locale ", here.id, " maxTaskPar=", here.maxTaskPar
   );
```

Listing 3.10: Wypisanie efektywnej liczby wątków dla każdego lokala.

3.2.4 Parametry konfiguracyjne

Zachowanie programu sterowane jest zbiorem stałych konfiguracyjnych `config const`, które można nadpisać z linii poleceń bez rekompilacji. Ich domyślne wartości (dla wariantu `3d.chpl`) zestawiono w tab. 3.2. Domyślny rozmiar sześcianu $20 \times 20 \times 20$ służy szybkiej weryfikacji; do eksperymentów wydajnościowych podnoszony jest do znacznie większych wartości. Parametr `alpha` to bezwymiarowy współczynnik dyfuzji wykorzystywany do obliczenia współczynników stencila, a `hotThickness` określa liczbę warstw wzdłuż osi *y* inicjalizowanych wartością 2,0 (gorąca warstwa startowa).

Tabela 3.2: Stałe konfiguracyjne programu i ich wartości domyślne (wariant `3d.chpl`).

Parametr	Wartość domyślna	Znaczenie
<code>nx, ny, nz</code>	20	wymiary siatki w trzech osiach
<code>numSteps</code>	100	liczba kroków czasowych
<code>alpha</code>	0,25	współczynnik dyfuzji
<code>hotThickness</code>	2	liczba gorących warstw wzdłuż osi <i>y</i>
<code>debug</code>	false	tryb diagnostyczny
<code>dumpDir</code>	frames	katalog zrzutów
<code>dumpEvery</code>	100	co ile kroków zapisywać klatkę
<code>compress</code>	true	kompresja zrzutów <code>gzip</code>
<code>trackMem</code>	false	śledzenie zużycia pamięci
<code>commLog</code>	false	rejestrowanie statystyk komunikacji

3.2.5 Rejestrowanie przebiegów

Sposób rejestrowania przebiegów zapewnia jednoczesny podgląd postępu, trwały zapis do dalszej analizy i jednoznaczne wiązanie logu z konfiguracją, w której powstał.

W tym celu wyjście uruchamianego programu przechwytywane jest poleceniem `tee` działającym po ustawieniu `set -o pipefail` (por. listing 3.3).

Nazwa pliku logu zawiera znacznik czasu w postaci `logs/<bin>-YYYYmmdd-HHMMSS.log`, przez co kolejne przebiegi tej samej binarki nie nadpisują się nawzajem i pozostają rozróżnialne w czasie.

3.3 Buforowanie, zapis i agregacja danych

3.3.1 Zapis fragmentów na dysk lokalny

Zapis stanu pola do dalszej wizualizacji zaprojektowano tak, aby ciężkie operacje wejścia-wyjścia nie obciążały ścieżki krytycznej obliczeń. Procedura `dumpLocal`, pokazana w listing 3.11, powoduje, że *każdy* lokal zapisuje wyłącznie własny podblok na lokalny system plików danego węzła; nie następuje żadne globalne zgromadzenie danych (*gather*) w jednym miejscu. Dzięki temu zapis jest w pełni rozproszony i skaluje się z liczbą węzłów.

```
1 proc dumpLocal(ref field, frameIdx: int) throws {
2   coforall loc in Locales do on loc {
3     const ld = field.localSubdomain();
4     const base = dumpDir + "/frame_" + frameIdx:string
5                 + "_loc_" + here.id:string + ".bin";
6     var block: [ld] real = field[ld];
7     // zapis naglowka i surowego bloku (opcjonalnie przez gzip)
8   }
9 }
```

Listing 3.11: Rozproszony zapis: każdy lokal zapisuje własny podblok lokalnie.

Powstający plik nosi nazwę `frame_<idx>_loc_<localeId>.bin` (lub `.bin.gz` przy kompresji) w katalogu `dumpDir`. Nagłówek zawiera indeks klatki, identyfikator lokala, sześć globalnych granic i sześć granic lokalnych, po których następuje surowy blok liczb typu `real`. Kompresja (domyślnie `compress=true`) realizowana jest przez przekierowanie strumienia do uruchomionego procesu `gzip -c > file.gz`. Ponieważ symulowane pole jest gładkie, kompresja `gzip` jest w praktyce bezstratna i zmniejsza rozmiar danych około 190-krotnie, co czyni częste zrzucanie klatek przystępnym pod względem zajętości dysku. Parametr `dumpEvery` pełni rolę bramki (`step % dumpEvery == 0`), a licznik klatek zwiększa się przy każdym faktycznym zrzucie. Przed rozpoczęciem pętli każdy lokal wykonuje `rmTree` i `mkdir` na katalogu `dumpDir`, aby wyeliminować pozostałości po wcześniejszych przebiegach. Błąd `ENOSPC` (numer 28) to problem dyskowy, inny niż sieciowy `ECONGESTION`, i to rozróżnienie bywa istotne przy diagnostyce nieudanych przebiegów.

3.3.2 Agregacja rozproszonych fragmentów

Ponieważ dane zapisywane są w postaci rozproszonych podbloków, konieczny jest osobny etap ich scalenia. Realizuje go jednolokalowy post-procesor `aggregate3d.chpl`. Dla każdej klatki alokuje on pełne, gęste pole, odnajduje wszystkie pliki `frame_<f>_loc_*.bin*` (rozpakowując warianty `.gz` poleceniem `gunzip -c`), a następnie odczytuje każdy podblok, którego nagłówek odzwierciedla format zapisany przez `dumpLocal`, i rozmieszcza go we właściwym miejscu pełnego pola. Rdzeń tej operacji pokazano w listing 3.12.

```
1 const blockDom = {lo[0]..hi[0], lo[1]..hi[1], lo[2]..hi[2]};
2 var block: [blockDom] real;
3 r.readBinary(block);
4 full[blockDom] = block;
```

Listing 3.12: Rekonstrukcja pełnego pola: odczyt podbloku i rozmieszczenie go we właściwym miejscu.

Po zrekonstruowaniu pełnego pola dla danej klatki wywoływana jest procedura `renderFrame(full)`. Podział na rozproszony zapis i scentralizowaną agregację jest zamierzony: komunikacja globalna odbywa się raz, w trybie post-processingu, i nie obciąża czasu obliczeń.

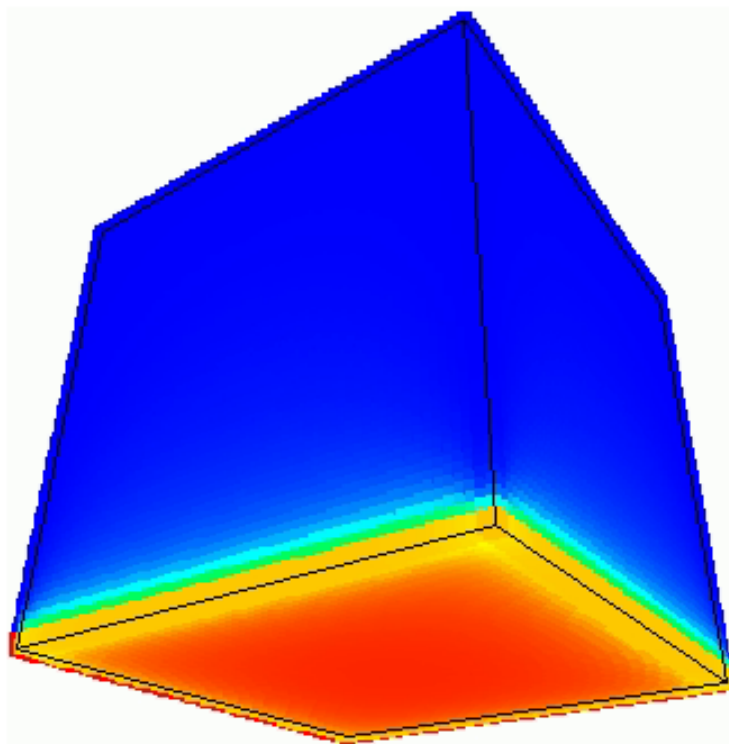
3.3.3 Wizualizacja

Ostatnim etapem, uruchamianym wyłącznie z parametrem `--render=true`, jest renderowanie zdefiniowane w module `ImageUtils.chpl`. Kolejne klatki kodowane są do pliku wideo (z użyciem `mediaPipe`, poprzez klatki w formacie BMP) o nazwie `movieName` (domyślnie `heat.mp4`). Zastosowano mapę barw HSV odwzorowującą ciepło od niebieskiego do czerwonego: odcień wyznaczany jest jako $240 \cdot t$ stopni, gdzie wartość 240 odpowiada zimnemu błękitowi, a 0, gorącej czerwieni.

Trójwymiarowy renderer wokseli rysuje wyłącznie sześć zewnętrznych ścian sześcienu (wnętrze jest pomijane) w rzucie perspektywicznym z obrotem (parametry `rotX`, `rotY`, `camDist`, `cubeScale`). Zaimplementowano usuwanie ścian tylnych (*back-face culling*), bufor głębi (Z-bufor), dylatację punktów (`pointSize`) oraz dwanaście czarnych krawędzi sześcienu kreślonych algorytmem Bresenhama. Widoczne ściany próbkowane są o jedną komórkę do wnętrza, ponieważ same woksele brzegowe nie są nigdy aktualizowane (stanowią stały warunek Dirichleta).

Sześcienn renderowany jest pod kątem, w widoku 3/4, od strony gorącej ściany $y = 0$. Ponieważ pięć ścian utrzymywanych jest w stałej temperaturze zimnej, a jedna w gorącej, render uwidacznia podtrzymywaną gorącą ścianę w kolorze czerwonym

oraz gradient temperatury na widocznych ścianach bocznych, opadający ku zimnym krawędziom. Wynik dla dziedziny 64^3 przedstawiono na rys. 3.3.



Rysunek 3.3: Render 3D pola temperatury dla dziedziny 64^3 , wykonany wbudowanym rendererem wokselowym programu z modułu `ImageUtils.chpl` i pokazany pod kątem od strony gorącej ściany. Widoczna jest podtrzymywana gorąca ściana $y = 0$ (czerwona, $T = 2$) oraz gradient na ścianach bocznych opadający ku zimnym ścianom ($T = 1$). Barwa odwzorowuje temperaturę w przedziale $[1, 2]$ zgodnie z mapą HSV od błękitu do czerwieni.

Opisana architektura, rozproszony zapis fragmentów bez globalnego gromadzenia, ich późniejsza agregacja poza ścieżką krytyczną oraz opcjonalna wizualizacja, oddziela operacje wejścia-wyjścia i komunikację zbiorową od właściwej pętli obliczeniowej. Jedynym elementem komunikacyjnym tej pętli pozostaje wymiana halo omówiona w podrozdz. 3.2.

Rozdział 4

Metodologia badań

4.1 Scenariusze i metryki

4.1.1 Scenariusze badawcze

Badania podzielono na dwie grupy: weryfikację poprawności oraz pomiary wydajności i skalowania. Weryfikacja poprawności obejmowała sprawdzenie, że wynik obliczeń jest niezależny od liczby lokacji, na które rozdzielono dziedzinę, pod warunkiem prawidłowej wymiany warstw brzegowych. Jako niezmiennik kontrolny wykorzystano sumę wartości pola w stanie końcowym, raportowaną przez program w wierszu `final field: min=... max=... sum=....`. Ponieważ pole przyjmuje wartości w przedziale $[1, 2]$, a jego średnia jest bliska jedności, suma jest w przybliżeniu równa liczbie komórek siatki.

Pomiary wydajności obejmowały cztery rodziny scenariuszy, zestawione w tab. 4.1:

- **skalowanie wątkowe w obrębie węzła:** jedna lokacja ($-n1\ 1$), zmienna liczba wątków $1/2/4/8/16$; przeprowadzone dla kostki 1000^3 na węźle Pionier-lab8-1 o 20 wątkach;
- **skalowanie wątkowe przy dziewięciu lokacjach:** kostka 1000^3 , ustalona liczba lokacji, zmienna liczba wątków na lokację;
- **skalowanie po liczbie węzłów:** kostka 1000^3 , liczba lokacji od 1 do 9;
- **skalowanie rozmiaru zadania:** dziewięć lokacji, krawędź kostki od 125 do 1000 (a więc od 125^3 do 1000^3 komórek).

Skalowanie wątkowe w obrębie węzła i skalowanie po węzłach rozdzielono świadomie: to dwa różne mechanizmy równoległości, współdzielenie pamięci przez wątki

Tabela 4.1: Rodziny scenariuszy pomiarowych. Liczbę wątków w pierwszej rodzinie oznaczono symbolem p , liczbę lokacji symbolem L , a krawędź kostki symbolem N .

Scenariusz	Zmienna	Ustalone	Środowisko
Wątki, 1000 ³	p $\{1, 2, 4, 8, 16\}$	$\in L=1, N=1000, 10$ kro- ków	węzeł Pionier-lab8-1 (20 wątków)
Wątki @ 9 loka- cji	p $\{1, 2, 4, 8, 16\}$	$\in L=9, N=1000, 100$ kro- ków	klaster (9 węzłów)
Węzły	$L \in \{1, \dots, 9\}$	$N=1000, 100$ kroków	klaster (9 węzłów)
Rozmiar kostki	N $\{125, \dots, 1000\}$	$\in L=9, 100$ kroków	klaster (9 węzłów)

jednej lokacji oraz rozproszenie danych między lokacjami połączone z komunikacją sieciową. Dopiero zestawienie obu perspektyw pozwala zidentyfikować czynnik ograniczający wydajność.

Wszystkie przebiegi wykorzystują ten sam zestaw parametrów numerycznych. Współczynnik dyfuzji przyjęto bezwymiarowo $\alpha = 0,25$, a gorącą warstwę startową o grubości `hotThickness=2` płaszczyzn inicjalizowano wzdłuż osi y przy ścianie $y = 0$. Krok czasowy dobierany jest automatycznie z warunku (2.6) ze współczynnikiem bezpieczeństwa $\Delta t = 0,98 \Delta t_{\max}$ (wzór (2.7)); dla największej badanej kostki $N = 1000$ daje to $\Delta t \approx 6,55 \cdot 10^{-7}$. Liczba kroków czasowych K zależy od rodziny scenariusza: wynosi 10 przy skalowaniu wątkowym na pojedynczym węźle oraz 100 w eksperymentach wielowęzłowych (patrz tab. 4.1).

4.1.2 Metryki

Podstawową wielkością mierzoną jest *czas wykonania* T , rozumiany jako czas trwania całej pętli czasowej, mierzony zegarem `computeTimer` i raportowany jako `Execution time`. Czas ten nie obejmuje inicjalizacji, czyli alokacji tablic, ustawienia warunku początkowego oraz obliczenia współczynników, mierzonej osobno zegarem `initTimer`. Dodatkowo, dla każdego kroku rejestrowano rozbieżność czasu na trzy składowe: czas wymiany warstw brzegowych (`updateFluff`), czas właściwych obliczeń stencilowych (`compute`) oraz czas zapisu klatki (`save`).

Na podstawie czasu wykonania zdefiniowano standardowe miary skalowania silnego [17]. Przyspieszenie przy p jednostkach równoległości, czyli wątkach albo lokacjach, wyraża się jako

$$S(p) = \frac{T(1)}{T(p)}, \quad (4.1)$$

gdzie $T(1)$ jest czasem wykonania konfiguracji odniesienia z jednym wątkiem albo

jedną lokacją. Wydajność równoległą zdefiniowano jako

$$E(p) = \frac{S(p)}{p} = \frac{T(1)}{pT(p)}, \quad (4.2)$$

przy czym w roli p dla skalowania wątkowego przyjęto *efektywną* liczbę wątków, tj. wartość `maxTaskPar` po ewentualnym ograniczeniu przez warstwę wątkową, a nie liczbę żadaną. Dla oceny wykorzystania sprzętu niezależnie od rozmiaru zadania posłużono się *przepustowością* wyrażoną w miliardach aktualizacji komórek na sekundę,

$$\Pi = \frac{N^3 \cdot K}{T \cdot 10^9}, \quad (4.3)$$

gdzie N^3 jest liczbą komórek, a K liczbą kroków czasowych [19]. Dla oszacowania roli komunikacji wprowadzono *udział komunikacji* w czasie pojedynczego kroku,

$$\phi = \frac{t_{\text{fluff}}}{t_{\text{fluff}} + t_{\text{compute}}}, \quad (4.4)$$

gdzie t_{fluff} i t_{compute} są uśrednionymi po krokach czasami wymiany halo oraz obliczeń. Wielkość ϕ przyjmuje wartości bliskie jedności, gdy dominuje komunikacja, oraz bliskie zeru, gdy dominują obliczenia. Uzupełniając rejestrowano objętość wymiany halo w bajtach na krok, obliczaną analitycznie przez procedurę `haloBytesPerStep` jako łączną liczbę komórek warstw brzegowych wszystkich lokacji pomnożoną przez rozmiar elementu.

4.2 Procedura i narzędzia pomiarowe

4.2.1 Procedura pomiarowa

Każdą konfigurację uruchamiano dziesięciokrotnie, a jako wartość reprezentatywną przyjmowano *medianę* czasu wykonania. Wybór mediany zamiast średniej arytmetycznej jest istotny: pojedyncze przebiegi bywają zaburzone przez chwilowe zdarzenia systemowe, takie jak rywalizacja o rdzenie, przejściowe ograniczenie taktowania procesora czy aktywność innych procesów, które zawyżają czas, lecz nie odzwierciedlają właściwości badanej konfiguracji. Mediana jest odporna na takie wartości odstające, podczas gdy średnia zostaje przez nie zawyżona. Przykładem jest jedna z serii jednowęzłowych, w której trzy z dziesięciu przebiegów były wolniejsze o około pół sekundy wskutek przejściowego ograniczenia taktowania; skutkowało to podwyższonym odchyleniem standardowym przy praktycznie niezmienionej medianie.

Przebiegi danej serii wykonywano sekwencyjnie, po jednym naraz, aby uniknąć wzajemnej rywalizacji o pamięć i pasmo. Wszystkie pomiary wydajności

prowadzono na środowisku Chapel 2.9 z warstwą komunikacyjną w wariacie `mpi-conduit`. Liczbę wątków na lokację ustawiano zmienną środowiskową `CHPL_RT_NUM_THREADS_PER_LOCALE`. Wyjście każdego przebiegu zapisywano jednocześnie na terminal i do pliku dziennika za pomocą polecenia `tee` z włączonym `set -o pipefail`, tak aby kod wyjścia potoku odzwierciedlał kod programu, a nie polecenia `tee`.

Konfigurację każdego przebiegu zapisywano bezpośrednio w dzienniku: skrypt uruchomieniowy umieszcza na początku logu wiersz `[cfg] threadsPerLocale(requested)=N numLocales=M` z żadaną liczbą wątków i lokacji, a sam program wypisuje, osobno dla każdej lokacji, wiersz `[cfg] locale <id> maxTaskPar=<n>` z *efektywną* liczbą wątków po ograniczeniu przez warstwę wątkową. Na węzłach Pionier o 20 wątkach sprzętowych liczba efektywna była równa żądanej dla wszystkich badanych konfiguracji ($p \leq 16$).

4.2.2 Narzędzia pomiarowe

Czasy mierzono wbudowanymi w język stoperami (`stopwatch`) o rozdzielczości podsekundowej, osobno dla inicjalizacji, całej pętli czasowej oraz, w każdym kroku, dla wymiany halo, obliczeń i zapisu. Objętość i liczbę operacji komunikacyjnych GET, PUT, zdalnych i atomowych rejestrowano za pomocą modułu diagnostyki komunikacji GASNet, włączanego przełącznikiem `commLog`; wypisywany raz wiersz `[comm]` podaje analityczną objętość wymiany halo na krok. Zużycie pamięci procesu kontrolowano narzędziem systemowym `/usr/bin/time -v`, odczytując maksymalny rezydentny zbiór stron, co pozwoliło ustalić graniczne rozmiary kostek mieszczące się w pamięci danego węzła.

Przetwarzanie zebranych dzienników i wykreślanie wyników zrealizowano skryptami w języku Python z biblioteką `matplotlib`. Podstawową formą prezentacji jest wykres pudełkowy rozkładu czasu wykonania z naniesionymi punktami poszczególnych przebiegów, medianą oraz średnią; uzupełniają go wykresy metryk pochodnych: przyspieszenia, wydajności, przepustowości i udziału komunikacji. Taka forma pokazuje jednocześnie środek rozkładu i rozrzut, co jest istotne przy ocenie powtarzalności.

Do diagnozy zachowania sieci wykorzystano dedykowany skrypt sondujący, który równocześnie obciąża łącze strumieniem `iperf3` w trybie UDP o przepływności zbliżonej do granicy łącza oraz monitoruje straty pakietów z kilku niezależnych perspektyw: procentu utraconych pakietów ICMP, przyrostów liczników odrzuceń na lokalnym interfejsie sieciowym oraz, co najważniejsze, przyrostów liczników błędów bufora odbiorczego UDP po stronie odbiorcy (`InErrors`, `RcvbufErrors`). Rosnące błędy bufora odbiorczego pod obciążeniem są bezpośrednim dowodem przeciążenia

typu *incast*, które w wariancie AMUDP prowadzi do awarii typu **ECONGESTION** [18]. Narzędzie to pozwoliło odróżnić przyczyny sieciowe od obliczeniowych i dyskowych oraz uzasadnić wybór wariantu transportu opartego na protokole TCP.

4.2.3 Przykład wyznaczenia metryk

Dla uporządkowania powyższych definicji poniżej prześledzono sposób wyznaczenia metryk pochodnych na jednym scenariuszu. Rozważono skalowanie wątkowe kostki 1000^3 na węźle Pionier o 20 wątkach. Konfiguracja odniesienia z jednym wątkiem osiągnęła medianę czasu wykonania $T(1) = 8,663\text{ s}$, natomiast konfiguracja czterowątkowa, $T(4) = 5,596\text{ s}$. Zgodnie z (4.1) przyspieszenie wynosi $S(4) = 8,663/5,596 \approx 1,548$, a wydajność równoległa według (4.2), $E(4) = 1,548/4 \approx 0,387$. Przepustowość dla tej konfiguracji, przy $K = 10$ krokach, obliczono z (4.3) jako $\Pi = 1000^3 \cdot 10 / (5,596 \cdot 10^9) \approx 1,79$ miliarda aktualizacji komórek na sekundę. Zabieg ten powtórzono dla każdej konfiguracji każdego scenariusza, dzięki czemu wszystkie wykresy i zestawienia w rozdz. 5 opierają się na jednolicie zdefiniowanych wielkościach.

4.2.4 Kontrola warunków i zagrożenia dla trafności

Wiarygodność wniosków zależy od kontroli czynników zakłócających. W toku badań zidentyfikowano i ograniczono kilka takich czynników.

Zmienność środowiska pochodzi ze współdzielenia maszyny z innymi procesami oraz dynamicznego zarządzania taktowaniem procesora, co wprowadza zmienność czasu wykonania. Ograniczono ją przez sekwencyjne uruchamianie przebiegów, dziesięciokrotne powtórzenia oraz stosowanie mediany. Tam, gdzie mimo to pojawiły się wartości odstające, czyli pojedyncze wolniejsze przebiegi wskutek przejściowego ograniczenia taktowania, fakt ten odnotowano jawnie i potraktowano jako artefakt pomiaru, a nie własność konfiguracji.

Nierówny podział dziedziny pojawia się, gdy liczba lokacji nie dzieli równo boku kostki 1000^3 ; powoduje to nierównomierne obciążenie i zaburza gładkość krzywej skalowania. Nie jest to błąd pomiaru, lecz rzeczywista własność dekompozycji; uwzględniono ją w interpretacji niemonotonicznego przebiegu skalowania po węzłach.

Trafność zewnętrzna wyników jest ograniczona: uzyskano je na konkretnym, niewielkim klastrze z siecią 1 Gb/s. Wartości bezwzględne nie przenoszą się wprost na inne środowiska, natomiast wnioski, ograniczenie przepustowością pamięci i komunikacją, korzystny wpływ większych zadań na wykorzystanie klastra, mają charakter ogólny i wynikają z właściwości samych obliczeń stencilowych [15, 16].

Rozdział 5

Wyniki i analiza

5.1 Poprawność i wydajność wariantów

5.1.1 Weryfikacja poprawności numerycznej

Przed przystąpieniem do jakichkolwiek pomiarów wydajnościowych konieczne było ustanowienie wiarygodnego kryterium poprawności obliczeń. Bez takiego kryterium przyspieszenie z równoległości nie ma znaczenia, łatwo bowiem uzyskać krótszy czas wykonania kosztem cichego naruszenia poprawności wyniku. W przypadku badanego programu przyjęto niezmiennik oparty na własnościach pola temperatury oraz na jego całkowitej sumie.

Warunki początkowe i brzegowe zadania dobrano tak, aby rozwiązanie pozostawało przez cały czas symulacji ograniczone do przedziału $[1, 2]$. Program po zakończeniu obliczeń raportuje statystyki końcowego pola w postaci wpisu `final field: min=1.0 max=2.0 sum=<S>`. Utrzymanie dolnej granicy 1,0 oraz górnej granicy 2,0 jest bezpośrednią konsekwencją zasady maksimum dla równania przewodnictwa cieplnego: schemat jawny z odpowiednio dobranym krokiem czasowym nie może wygenerować wartości spoza zakresu wyznaczonego przez dane początkowe i brzegowe. Naruszenie któregoś z tych ograniczeń natychmiast sygnalizowałoby błąd implementacji schematu różnicowego lub niestabilność numeryczną.

Kluczową rolę w weryfikacji odgrywa jednak suma wszystkich elementów pola. Ponieważ średnia wartość pola wynosi w przybliżeniu 1, suma S jest bliska liczbie węzłów siatki, czyli N^3 dla dziedzin o boku N . Jednocześnie suma S dobrze oddaje stan całej dziedziny i szybko reaguje na różnice między konfiguracjami: najmniejsze rozejście się wyników widać jako rozbieżność sum.

Właśnie na tej podstawie zweryfikowano poprawność obliczeń wielowęzłowych. Przebieg uruchomiony na pojedynczym locale (`-nl 1`) daje pewną wartość sumy odniesienia. Ten sam problem rozdzielony na N locale daje identyczną sumę *pod*

warunkiem, że po każdym kroku czasowym wykonywana jest wymiana warstw brzegowych (halo) za pomocą operacji `updateFluff`. Pominiecie tej wymiany prowadzi do zauważalnego skrócenia czasu wykonania, ponieważ znika najkosztowniejsza faza komunikacyjna, lecz suma S zaczyna się rozchodzić względem wartości odniesienia. Wynik jest wtedy *cicho błędny*: program kończy się bez żadnego komunikatu o błędzie, a mimo to obliczone pole nie odpowiada rozwiązaniu zadania. Wymiana warstw brzegowych jest więc potrzebna do poprawności obliczeń wielowęzłowych i nie da się jej pominąć dla zysku wydajności. Ta obowiązkowa komunikacja, jak pokazano w podrozdz. 5.2, jest głównym wąskim gardłem systemu.

5.1.2 Skalowanie wątkowe na pojedynczym węźle

W tej części przeanalizowano wydajność jednowęzłową. Celem tej serii eksperymentów było zbadanie, jak czas wykonania programu zależy od liczby wątków obliczeniowych przy ustalonym rozmiarze zadania, a zatem ile faktycznie zyskuje się na zrównolegleniu wewnątrzwęzłowym. Pomiary wykonano dla dziedziny 1000^3 na węźle klastra Pionier (12 rdzeni / 20 wątków, 32 GiB pamięci).

Tabela 5.1 przedstawia wyniki tego eksperymentu; faktyczna liczba zadań qth-reads jest tu równa liczbie żądanej.

Tabela 5.1: Skalowanie wątkowe na pojedynczym węźle, dziedzina 1000^3 , -n1 1, 10 kroków $\times$ 10 powtórzeń, Chapel 2.9, conduit MPI, węzeł Pionier.

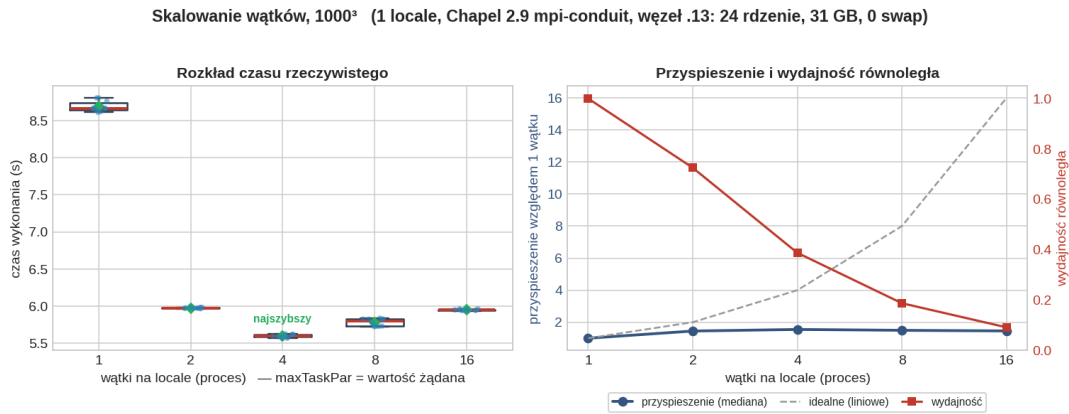
Żądane wątki	maxTaskPar	Mediana [s]	Średnia [s]	Min. [s]	Maks. [s]	Odch. std. [s]	S	E
1	1	8,663	8,688	8,608	8,802	0,069	1,000	1,000
2	2	5,976	5,976	5,964	5,985	0,006	1,450	0,725
4	4	5,596	5,598	5,568	5,627	0,020	1,548	0,387
8	8	5,798	5,782	5,724	5,839	0,047	1,494	0,187
16	16	5,950	5,950	5,938	5,965	0,009	1,456	0,091

Obraz jest tu jakościowo odmienny i znacznie zdrowszy. Przejście z jednego do dwóch wątków przynosi wyraźny skok wydajności: czas spada z 8,663s do 5,976s, co odpowiada przyspieszeniu 1,450. Optimum osiągane jest przy czterech wątkach (mediana 5,596s, przyspieszenie 1,548), po czym dalsze zwiększanie liczby wątków ponownie zaczyna szkodzić: przy ośmiu wątkach czas rośnie do 5,798s, a przy szesnastu do 5,950s. Maksymalne uzyskane przyspieszenie wynosi zatem tylko około $1,55\times$, mimo dostępności dwudziestu wątków.

Wydajność E spada przy tym bardzo ostro: z 1,000 przez 0,725 i 0,387 aż do 0,091 przy szesnastu wątkach. Tak gwałtowny spadek to typowy objaw problemu *ograniczonego pasmem pamięci* [16]. Schemat różnicowy jest operacją o niskiej in-

tensywności arytmetycznej, ponieważ na każdy odczytany element siatki przypada niewiele operacji zmiennoprzecinkowych, przez co jego przepustowość jest limitowana szybkością, z jaką procesor jest w stanie pobierać dane z pamięci głównej, a nie liczbą dostępnych rdzeni. Po nasyceniu pasma pamięci dodatkowe wątki nie mają czym się zająć poza rywalizacją o tę samą magistralę, co prowadzi do degradacji wydajności.

Zwiększanie liczby wątków ponad pewien próg nie pomaga tu, a wręcz szkodzi: po nasyceniu magistrali pamięci dodatkowe wątki nie znajdują użytecznej pracy. Rysunek 5.1 ilustruje te zależności: lewy panel przedstawia rozkład czasów wykonania w postaci wykresu pudełkowego, a prawy zestawia zmierzone przyspieszenie z przyspieszeniem idealnym oraz z krzywą wydajności, uwidaczniając odejście od skalowania liniowego już przy dwóch wątkach.



Rysunek 5.1: Skalowanie wątkowe dla dziedziny 1000^3 na węźle Pionier. Lewy panel: rozkład czasu wykonania w postaci wykresu pudełkowego w funkcji liczby wątków. Prawy panel: zmierzone przyspieszenie na tle przyspieszenia idealnego oraz wydajność E .

W części jednowęzłowej równoległość wątkowa przynosi w najlepszym razie skromny, kilkudziesięcioprocentowy zysk, a wydajność szybko załamuje się wraz ze wzrostem liczby wątków. Bezpośrednią przyczyną jest ograniczenie pasmem pamięci: schemat różnicowy o niskiej intensywności arytmetycznej nasycza magistralę pamięci przy niewielkiej liczbie wątków. Sytuacja ta pogłębia się dopiero przy skalowaniu na wiele węzłów.

5.2 Skalowanie i ograniczenia komunikacji

W tej części analizuje się zachowanie programu przy rozdzieleniu obliczeń na wiele węzłów klastra połączonych siecią Ethernet 1 Gb/s. Eksperymenty w tej sekcji przeprowadzono na klastrze dla dziedziny 1000^3 i 100 kroków czasowych. Wykazano, że badany system jest *ograniczony komunikacją*: to przepustowość sieci, a nie moc

obliczeniowa węzłów, wyznacza pułap osiąganą wydajności.

5.2.1 Skalowanie liczbą węzłów

Tabela 5.2 przedstawia czas wykonania w funkcji liczby węzłów, od jednego do dziewięciu, dla dziedziny 1000^3 . Oprócz statystyk czasu podano przyspieszenie, wydajność oraz objętość danych wymienianych w ramach warstw brzegowych na jeden krok czasowy w kolumnie „bajty/krok”.

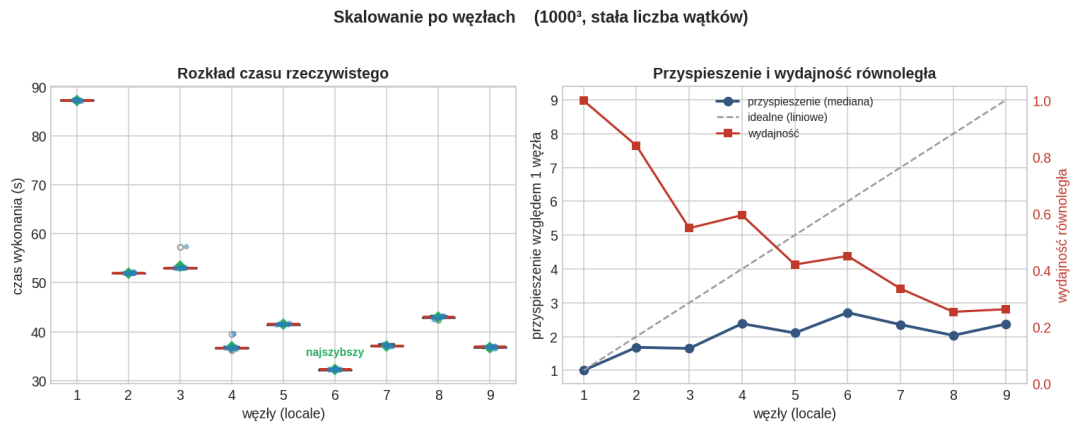
Tabela 5.2: Skalowanie liczbą węzłów, dziedzina 1000^3 , 100 kroków.

Węzły	Mediana [s]	Średnia [s]	Min. [s]	Odch. std. [s]	S	E	Bajty/krok
1	87,344	87,307	87,104	0,092	1,000	1,000	0
2	51,943	51,957	51,858	0,107	1,682	0,841	16000000
3	52,978	53,413	52,862	1,316	1,649	0,550	32000000
4	36,670	36,889	36,203	0,894	2,382	0,595	32032000
5	41,476	41,482	41,347	0,093	2,106	0,421	64000000
6	32,240	32,243	32,076	0,115	2,709	0,452	48064000
7	37,114	37,112	36,909	0,156	2,353	0,336	96000000
8	42,941	42,909	42,419	0,198	2,034	0,254	48096064
9	36,832	36,817	36,469	0,157	2,371	0,263	64128000

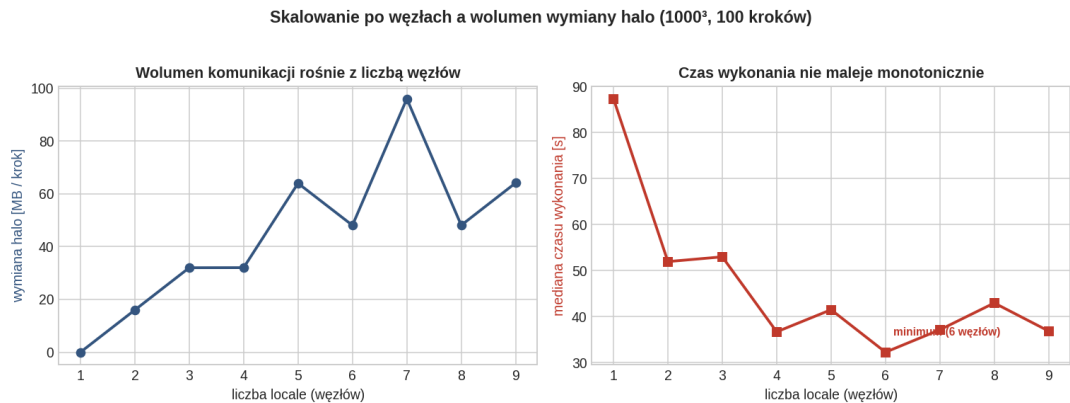
Skalowanie węzłowe okazuje się wyraźnie *niemonotoniczne*. Zamiast oczekiwanego, systematycznego spadku czasu wraz ze wzrostem liczby węzłów, obserwuje się silny „ząbek” pomiędzy parzystymi a nieparzystymi liczbami węzłów. Najlepszy wynik uzyskano dla sześciu węzłów (mediana 32,240 s, przyspieszenie 2,709), podczas gdy sąsiednie konfiguracje pięcio- i siedmiowęzłowa wypadają zauważalnie gorzej (41,476 s oraz 37,114 s). Źródłem tego zjawiska jest nierównomierny podział dziedziny 1000^3 : gdy liczba węzłów nie dzieli gładko boku kostki, powstaje nieproporcjonalne obciążenie, w którym niektóre węzły otrzymują większe poddziedziny i stają się elementem spowalniającym całość, co dodatkowo splata się ze wzrostem objętości komunikacji między węzłami.

Niezależnie od tych oscylacji, obraz ogólny jest jednoznaczny: przyspieszenie *plaszczy się* na poziomie około $2,4\times$ i nie zbliża się do skalowania liniowego, mimo dziewięciokrotnego zwiększenia liczby węzłów [17]. Wydajność spada z 1,000 do zaledwie 0,263 przy dziewięciu węzłach. Kluczem do wyjaśnienia jest ostatnia kolumna tabeli: objętość wymienianych warstw brzegowych rośnie w przybliżeniu liniowo wraz z liczbą węzłów, od 0 bajtów dla pojedynczego węzła, w którym nie ma komunikacji międzywęzłowej, do ponad 64000000 bajtów na krok przy dziewięciu węzłach. Każdy dodany węzeł wnosi pewną moc obliczeniową, lecz zarazem powiększa całko-

wity ruch sieciowy, a przy powolnym łączu ten drugi efekt szybko pochłania korzyść z pierwszego. System jest zatem ograniczony komunikacją.



Rysunek 5.2: Skalowanie liczbą węzłów dla dziedziny 1000³. Lewy panel: rozkład czasu wykonania w funkcji liczby węzłów. Prawy panel: przyspieszenie na tle przyspieszenia idealnego oraz wydajność E .



Rysunek 5.3: Objętość warstw brzegowych wymienianych na krok w megabajtach oraz mediana czasu wykonania w funkcji liczby węzłów.

Zależności te zilustrowano na rys. 5.2 oraz rys. 5.3. Pierwsza z nich uwidacznia zarówno oscylacyjny charakter czasu wykonania, jak i znaczne odejście krzywej przyspieszenia od ideału liniowego. Druga zestawia rosnącą objętość komunikacji z czasem wykonania, czyniąc bezpośrednim widocznym związek między nasilającym się ruchem sieciowym a stagnacją wydajności.

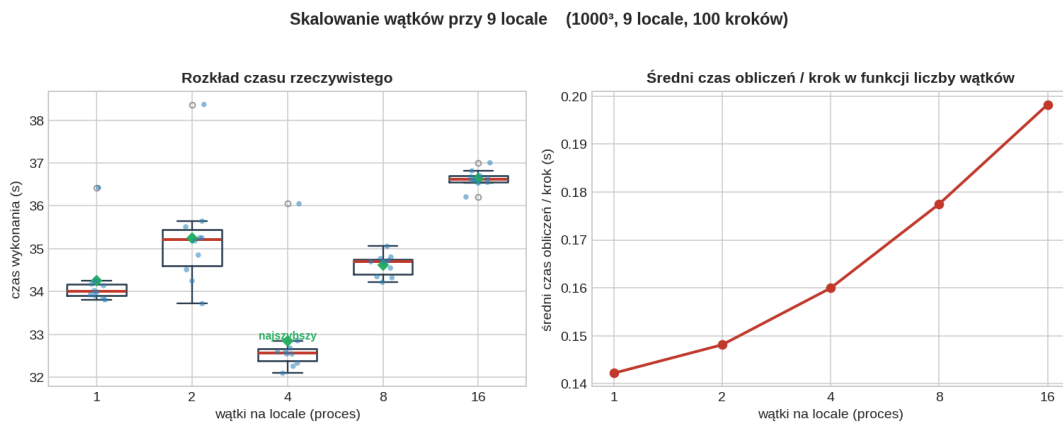
5.2.2 Skalowanie wątkowe przy dziewięciu węzłach

Aby rozstrzygnąć, czy wąskim gardłem w konfiguracji wielowęzłowej jest komunikacja, czy może niedostateczne wykorzystanie rdzeni w obrębie węzła, powtórzono skalowanie wątkowe przy ustalonej liczbie dziewięciu węzłów. Wyniki zawiera tab. 5.3.

Tabela 5.3: Skalowanie wątkowe przy dziewięciu węzłach, dziedzina 1000^3 , 100 kroków.

Konfiguracja	Mediana [s]	Średnia [s]	Min. [s]	Odch. std. [s]	S	Obl./krok [s]
1 wątek	34,003	34,245	33,805	0,740	1,000	0,1422
2 wątki	35,217	35,252	33,715	1,184	0,966	0,1481
4 wątki	32,563	32,850	32,090	1,086	1,044	0,1600
8 wątków	34,694	34,616	34,215	0,245	0,980	0,1775
16 wątków	36,623	36,633	36,208	0,195	0,928	0,1982

Odpowiedź jest jednoznaczna. Czas wykonania pozostaje niemal płaski w zakresie od około 32 do 37 sekund niezależnie od liczby wątków, z płytkim minimum przy czterech wątkach (32,563 s). Przyspieszenie oscyluje wokół jedności i nigdy nie przekracza 1,044. Najbardziej wymowna jest jednak kolumna średniego czasu obliczeń na krok: rośnie ona *monotonicznie* wraz z liczbą wątków, od 0,1422 s przy jednym wątku do 0,1982 s przy szesnastu. Gdyby problem był ograniczony mocą obliczeniową, dodawanie wątków skracałoby czas obliczeń na krok; obserwuje się jednak zjawisko przeciwne: czas ten rośnie. Świadczy to o *nadmiernej subskrypcji* (oversubscription): dodatkowe wątki nie znajdują użytecznej pracy, lecz rywalizują o pasmo pamięci oraz konkurują z wątkiem odpytującym warstwę komunikacyjną GASNet [7]. Zwiększanie liczby wątków nie pomaga, ponieważ problem jest ograniczony komunikacją, a nie obliczeniami. Rysunek 5.4 przedstawia te zależności graficznie.



Rysunek 5.4: Skalowanie wątkowe przy dziewięciu węzłach, dziedzina 1000^3 . Lewy panel: rozkład czasu wykonania. Prawy panel: średni czas obliczeń na krok w funkcji liczby wątków, rosnący wskutek nadmiernej subskrypcji.

5.2.3 Skalowanie rozmiaru zadania

Ostatni zestaw eksperymentów bada wpływ rozmiaru dziedziny na wydajność przy ustalonej liczbie dziewięciu węzłów. Tabela 5.4 zestawia czasy wykonania, przepustowość wyrażoną w miliardach komórek na sekundę (mld komórek/s) oraz udział komunikacji `comm_frac`, zdefiniowany jako iloraz czasu wymiany warstw brzegowych i sumy czasu wymiany oraz czasu obliczeń.

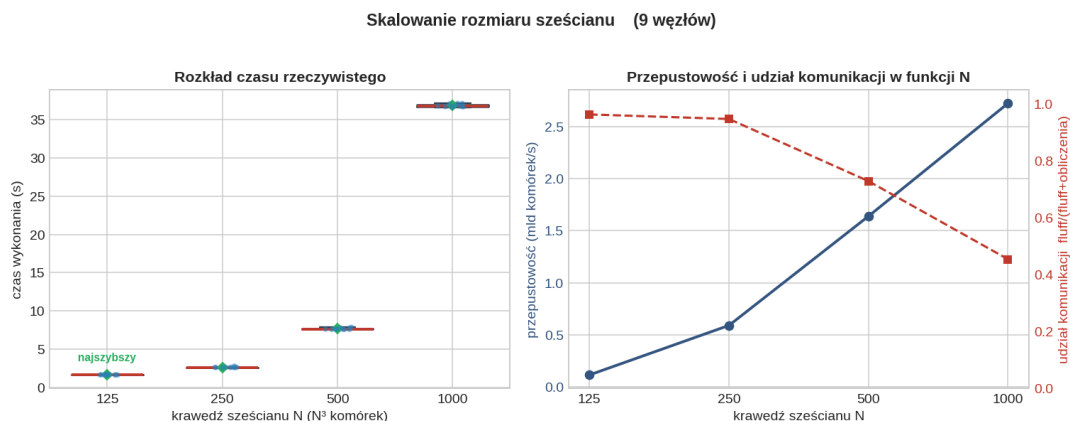
Tabela 5.4: Skalowanie rozmiaru dziedziny przy dziewięciu węzłach, 100 kroków.

Rozmiar	Mediana [s]	Średnia [s]	Przepustowość [mld komórek/s]	<code>comm_frac</code>
125^3	1,668	1,670	0,117	0,963
250^3	2,640	2,652	0,592	0,947
500^3	7,625	7,673	1,639	0,729
1000^3	36,741	36,802	2,722	0,454

Wynik ten najlepiej potwierdza, że ograniczenie ma charakter komunikacyjny. Przepustowość rośnie systematycznie wraz z rozmiarem dziedziny, od zaledwie 0,117 mld komórek/s dla najmniejszej kostki 125^3 do 2,722 mld komórek/s dla największej kostki 1000^3 . Jednocześnie udział komunikacji `comm_frac` maleje monotonicznie z 0,963 do 0,454. Innymi słowy, dla małych dziedzin blisko dziewięćdziesiąt sześć procent czasu pochłania komunikacja, a faktyczne obliczenia stanowią margines; wraz ze wzrostem dziedziny proporcje te wyrównują się i przy 1000^3 komunikacja oraz obliczenia zajmują już zbliżony czas. Nawet przy największej zbadanej kostce komunikacja wciąż odpowiada za blisko połowę czasu kroku; trend jest wprawdzie korzystny, lecz w osiągalnym zakresie rozmiarów zadanie nie przestaje być istotnie obciążone komunikacją.

Mechanizm tego zjawiska jest geometryczny. Objętość dziedziny rośnie jak N^3 , podczas gdy powierzchnia warstw brzegowych, decydująca o ilości komunikacji, rośnie jedynie jak N^2 . W konsekwencji wraz ze wzrostem N stosunek pracy obliczeniowej do komunikacji poprawia się, a duże kostki znacznie lepiej wykorzystują dostępny klaster niż małe, które są całkowicie zdominowane przez narzut komunikacyjny. Pokazuje to w praktyce znany fakt, że efektywne skalowanie na maszynach rozproszonych wymaga, by na każdy węzeł przypadało dostatecznie dużo pracy obliczeniowej.

Największą zbadaną dziedziną jest 1000^3 ; dziedziny istotnie większe przekraczają możliwości pamięciowe i czasowe klastra, co wyznacza praktyczną, górną granicę rozmiaru zadania osiągalnego na badanej infrastrukturze. Omówione zależności ilustruje rys. 5.5.



Rysunek 5.5: Skalowanie rozmiaru dziedziny przy dziewięciu węzłach. Lewy panel: rozkład czasu wykonania dla kolejnych rozmiarów. Prawy panel: przepustowość oraz udział komunikacji `comm_frac` w funkcji rozmiaru kostki.

5.2.4 Synteza: system ograniczony komunikacją

Wyniki z tego rozdziału wskazują wspólnie, że system jest ograniczony komunikacją i przepustowością pamięci, a nie mocą obliczeniową. Przyspieszenie węzłowe płaszczy się na poziomie około $2,4\times$ (tab. 5.2), daleko od skalowania liniowego, mimo dziewięciokrotnego zwiększenia liczby węzłów. Zwiększanie liczby wątków na węzeł nie przynosi poprawy, a w konfiguracji wielowęzłowej średni czas obliczeń na krok wręcz rośnie z liczbą wątków (tab. 5.3), co wskazuje na nadmierną subskrypcję, a nie na niedobór mocy obliczeniowej. Udział komunikacji dominuje przy małych dziedzinach i maleje dopiero dla dużych kostek (tab. 5.4), zgodnie z geometryczną relacją między objętością N^3 a powierzchnią N^2 . Na każdym węźle jeden wątek warstwy komunikacyjnej GASNet obciąża rdzeń w niemal stu procentach, prowadząc aktywne odpytywanie (busy-polling) w oczekiwaniu na postęp komunikacji, co potwierdzono analizą stosów wywołań w debugerze. To nie jest obciążenie obliczeniowe, lecz widoczny i mierzalny skutek ograniczenia komunikacją.

Fizycznym ograniczeniem jest łącze Ethernet o przepustowości 1 Gb/s, które dla dziedziny rzędu 1000^3 ulega nasyceniu. Objętość warstw brzegowych na krok rośnie zarówno z liczbą węzłów, jak i z kwadratem boku dziedziny, wobec czego to sieć, a nie procesory, wyznacza pułap wydajności.

Osobnego omówienia wymaga dobór konduitu komunikacyjnego GASNet, który okazał się decydujący dla tego, czy obliczenia w ogóle kończą się poprawnie [6]. Konduit `udp` (AMUDP) przerywa działanie z błędem `ECONGESTION` w warunkach ruchu typu wielu nadawców do jednego odbiorcy (incast) oraz przy utracie pakietów. Natomiast konduit `mpi`, oparty na implementacji MPICH i korzystający z kontroli przepływu protokołu TCP, przetrwa te warunki i działa wolniej, ale kończy obliczenia poprawnie. Z tego właśnie powodu nowsza seria pomiarów wykorzystuje

konduit MPI. To kompromis: conduit MPI jest wolniejszy, ale działa poprawnie w zawodnej sieci, co jest tu ważniejsze od samej szybkości.

Skoro system jest ograniczony komunikacją, dalszej poprawy wydajności nie należy szukać ani w zwiększaniu liczby wątków, ani w dokładaniu kolejnych węzłów, lecz przede wszystkim w przyspieszeniu warstwy komunikacyjnej, czy to przez zastosowanie szybszego łącza sieciowego, czy przez wybór wydajniejszego konduitu GANet. Dopóki halo wymieniane jest przez łącze 1 Gb/s, klaster wciąż czeka na dane z sieci, a część rdzeni pozostaje niewykorzystana.

5.2.5 Zestawienie zbiorcze i powtarzalność

Dla wygody czytelnika zebrano najważniejsze wnioski ilościowe z poszczególnych rodzin eksperymentów w jednym miejscu. Tabela 5.5 podaje dla każdej serii kluczową miarę oraz jej najistotniejszy wynik.

Tabela 5.5: Zestawienie zbiorcze najważniejszych wyników eksperymentalnych rozdziału.

Rodzina eksperymentów	Kluczowa miara	Wynik
Wątki 1000 ³ (tab. 5.1)	optimum / maks. przyspieszenie	4 wątki; 1,55×
Węzły 1000 ³ (tab. 5.2)	optimum / plateau przyspieszenia	6 węzłów; ok. 2,4×
Wątki, 9 węzłów (tab. 5.3)	przyspieszenie (płaskie)	≤ 1,044
Rozmiar (tab. 5.4)	kostki przepustowość	0,117–2,722 mld komórek/s
Rozmiar (tab. 5.4)	kostki udział komunikacji <code>comm_frac</code>	0,963→0,454

Z zestawienia widać, że niezależnie od osi skalowania, liczby wątków, liczby węzłów czy rozmiaru zadania, wyniki trafiają na to samo ograniczenie: pasmo pamięci i komunikację. Jediną osią, wzdłuż której obserwuje się wyraźną poprawę efektywności wykorzystania klastra, jest rozmiar zadania: powiększanie kostki przesuwając proporcje na korzyść obliczeń, zgodnie z omówioną wcześniej relacją N^3 do N^2 .

Odrębnego komentarza wymaga powtarzalność pomiarów, stanowiąca podstawę zaufania do prezentowanych median. Zdecydowana większość serii charakteryzuje się bardzo niskim rozrzutem wyników. Dla serii 1000³ na węźle Pionier (tab. 5.1) odchylenie standardowe czasu wykonania nie przekracza 0,07 s, a dla większości konfiguracji jest rzędu zaledwie kilku tysięcznych sekundy. Tak niski rozrzut oznacza, że pojedynczy przebieg jest w tych przypadkach niemal równie miarodajny co mediana z dziesięciu powtórzeń, a różnice między konfiguracjami to rzędy dziesiątych

sekundy lub więcej, więc nie da się ich wytłumaczyć samym szumem pomiarowym.

Osobną kwestią jest praktyczna granica rozmiaru zadania. Dziedzin istotnie większych niż 1000^3 nie udało się ukończyć: przekraczają one możliwości pamięciowe i czasowe klastra, wobec czego nie figurują w żadnej tabeli i wyznaczają jedynie górny kres rozmiaru osiągalnego na badanej infrastrukturze. Świadomość tego ograniczenia nie podważa głównych wniosków rozdziału, które opierają się na seriach o niskim rozrzucie i dziesięciokrotnej powtarzalności, lecz trzeba o niej pamiętać, oceniając, do jakich rozmiarów zadania sięgają wnioski tej pracy.

Rozdział 6

Podsumowanie i wnioski

6.1 Wnioski

Celem pracy było zaprojektowanie, zaimplementowanie i wdrożenie na klastrze rozproszonego programu symulacyjnego rozwiązującego trójwymiarowe równanie przewodnictwa ciepła w języku Chapel oraz przeprowadzenie systematycznej analizy jego poprawności i skalowalności. Zrealizowano wszystkie założone cele, a wyniki potwierdzają tezę pracy: dla rozważanej klasy obliczeń stencilowych o czynniku ograniczającym wydajność decyduje nie moc obliczeniowa, lecz przepustowość pamięci na poziomie węzła oraz koszt komunikacji między węzłami [15].

W zakresie poprawności wykazano, że rozwiązanie różnicowe daje wynik niezależny od liczby lokacji, na które rozdzielono dziedzinę, pod warunkiem prawidłowej wymiany warstw brzegowych; niezmiennik sumy pola w stanie końcowym okazał się prostym, lecz skutecznym kryterium kontrolnym. Pominięcie wymiany halo znacząco skraca czas wykonania, lecz prowadzi do wyniku *cicho błędnego*, co pokazuje, że w programowaniu rozproszonym krótki czas wykonania nie oznacza jeszcze poprawnego wyniku.

Zrównoleglenie wątkowe w obrębie węzła nasycza się bardzo szybko: dla kostki 1000^3 optimum przypada na cztery wątki, a maksymalne przyspieszenie nie przekracza około $1,55\times$ mimo szesnastu dostępnych wątków, co jest sygnaturą ograniczenia przepustowością pamięci. Skalowanie po węzłach zatrzymuje się na poziomie około $2,4\times$ i pozostaje daleko od zależności liniowej, a jego przebieg jest niemonotoniczny, z najlepszym czasem przy sześciu węzłach, wskutek nierównego podziału kostki 1000^3 dla „niewygodnych” dzielników oraz rosnącej objętości wymiany halo; przy łączu 1 Gb/s czyni to obliczenia ograniczonymi komunikacją. Poprawę przynosi dopiero powiększanie zadania: wraz ze wzrostem krawędzi kostki przepustowość rośnie z 0,117 do 2,722 mld komórek na sekundę, a udział komunikacji maleje z 0,96 do 0,45, ponieważ objętość rośnie z sześcianem krawędzi, a powierzchnia wymiany

halo jedynie z jej kwadratem.

Praktyczne znaczenie miał dobór conduitu GASNet. Wariant AMUDP, choć prosty, ulega awarii typu *ECONGESTION* pod obciążeniem typu *incast* i przy stratach pakietów, bo nie zapewnia kontroli przepływu [18]; *mpi-conduit* oparty na bibliotece MPICH i protokole TCP przetrwa te warunki dzięki kontroli przepływu i retransmisji, kosztem nieco niższej wydajności. Wyższa warstwa języka również miała znaczenie: globalny widok tablic, gotowy rozkład *StencilDist* z automatyczną wymianą otoczek oraz zwięzłe iteratory równoległe pozwoliły zapisać poprawny kod rozproszony znacznie prościej niż przy jawnym użyciu MPI [1, 2]. Abstrakcja ta nie zniosła jednak fundamentalnego ograniczenia komunikacyjnego: koszt wymiany halo pozostaje widoczny w wynikach i wyznacza granice skalowania. Model programowania nie zmienia właściwości sieci, co w tym przypadku oznacza, że produktywność i wydajność się uzupełniają, a nie zastępują.

6.2 Ograniczenia

Uzyskane wyniki należy interpretować z uwzględnieniem kilku ograniczeń.

Środowiskiem docelowym był niewielki, jednorodny klaster połączony siecią Ethernet o przepustowości 1 Gb/s, która jest wąskim gardłem i w znacznej mierze determinuje granice skalowania po węzłach. Na infrastrukturze z szybszą siecią o małych opóźnieniach, na przykład InfiniBand, można by oczekiwać lepszego skalowania oraz przesunięcia punktu, w którym program przestaje być ograniczony komunikacją, choć weryfikacja tej hipotezy wymagałaby odpowiedniego sprzętu. Praktyczną granicą jest też maksymalny rozmiar zadania: dziedzin istotnie większych niż 1000^3 nie udało się zmieścić w pamięci i czasie klastra.

Zakres pracy świadomie zawężono do jawnej metody różnic skończonych na siatce regularnej z arytmetyką podwójnej precyzji. Metody niejawne, które są bezwarunkowo stabilne, lecz wymagają rozwiązywania układów równań, siatki niestrukturalne, techniki nakładania komunikacji na obliczenia oraz akceleracja na procesorach graficznych pozostają poza zakresem, a każde z nich mogłoby zmienić charakterystykę obliczeniową problemu. Wreszcie badania dotyczyły wyłącznie skalowania silnego, przy ustalonym rozmiarze zadania i rosnącej liczbie jednostek równoległości; skalowanie słabe, gdzie rozmiar zadania rośnie proporcjonalnie do liczby lokacji, dałoby obraz uzupełniający.

6.2.1 Kierunki dalszych prac

Te ograniczenia wyznaczają naturalne kierunki dalszych prac.

Najbardziej obiecującym z nich jest nakładanie komunikacji na obliczenia: wymianę warstw brzegowych można rozpocząć asynchronicznie i w tym czasie liczyć wewnątrz poddziedziny, które nie zależą od danych sąsiadów, ujawniając komunikację dopiero przy aktualizacji komórek przylegających do granicy. W języku Chapel wyraża się to naturalnie, za pomocą zadań asynchronicznych i jawnego rozdzielania wnętrza od brzegu poddziedziny.

Powtórzenie pomiarów na sieci o niskim opóźnieniu i wysokiej przepustowości, na przykład InfiniBand, wraz z conduitem GASNet dobranym do takiej sieci pozwoliłoby ustalić, w jakim stopniu obserwowane granice skalowania wynikają z samego algorytmu, a w jakim z ograniczeń konkretnego łącza 1 Gb/s. Uzupełniając warto rozważyć skalowanie słabe, gdzie rozmiar zadania rośnie proporcjonalnie do liczby lokacji, co pozwoliłoby oddzielić koszty stałe od kosztów rosnących wraz z rozmiarem problemu.

Innym naturalnym rozszerzeniem są warianty numeryczne i sprzętowe: schematy niejawne, które są bezwarunkowo stabilne, lecz wymagają rozwiązywania układów równań i globalnej komunikacji innego typu, siatki niestrukturalne oraz akceleracja jądra obliczeniowego na procesorach graficznych, którą Chapel wspiera odpowiednimi lokacjami i atrybutami pętli równoległych. Każde z nich zmieniałoby charakterystykę obliczeniową problemu i pozwoliłoby sprawdzić, na ile wnioski tej pracy zachowują ważność poza klasą jawnych schematów stencilowych.

Z kolei inżynierskim uzupełnieniem byłoby pełne zautomatyzowanie potoku pomiarowego, od budowy i dystrybucji po zbieranie logów i generowanie wykresów, tak aby cały cykl badawczy był odtwarzalny jednym poleceniem. Wprowadzone w tej pracy elementy, czyli samoopisujące się logi oraz skrypty dystrybucji i analizy, mogą być bazą takiego potoku.

Bibliografia

- [1] B. L. Chamberlain, D. Callahan, H. P. Zima, *Parallel Programmability and the Chapel Language*, International Journal of High Performance Computing Applications, 21(3), 2007, s. 291–312.
- [2] B. L. Chamberlain, *Chapel*, w: P. Balaji (red.), *Programming Models for Parallel Computing*, MIT Press, 2015, s. 129–159.
- [3] The Chapel Team, *Chapel Language Specification and User Documentation*, wersja 2.9, Chapel a Series of LF Projects, LLC, 2026. <https://chapel-lang.org>.
- [4] The Chapel Team, *The Stencil Distribution (**StencilDist**): Standard Module Documentation*, 2026. <https://chapel-lang.org/docs/modules/dists/StencilDist.html>.
- [5] K. B. Wheeler, R. C. Murphy, D. Thain, *Qthreads: An API for Programming with Millions of Lightweight Threads*, IEEE International Symposium on Parallel and Distributed Processing (IPDPS), 2008.
- [6] D. Bonachea, P. H. Hargrove, *GASNet-EX: A High-Performance, Portable Communication Library for Exascale*, Languages and Compilers for Parallel Computing (LCPC 2018), LNCS 11882, Springer, 2019.
- [7] D. Bonachea, *GASNet Specification*, Technical Report UCB/CSD-02-1207, University of California, Berkeley, 2002 (rewidowana do wersji 1.8).
- [8] G. Almasi, *PGAS (Partitioned Global Address Space) Languages*, w: *Encyclopedia of Parallel Computing*, Springer, 2011, s. 1539–1545.
- [9] UPC Consortium, *UPC Language Specifications, v1.3*, Lawrence Berkeley National Laboratory, LBNL-6623E, 2013.
- [10] W. Gropp, *MPICH2: A New Start for MPI Implementations*, Recent Advances in PVM and MPI, LNCS 2474, Springer, 2002.

- [11] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023.
- [12] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*, SIAM, Philadelphia, 2007.
- [13] J. C. Strikwerda, *Finite Difference Schemes and Partial Differential Equations*, wyd. 2, SIAM, Philadelphia, 2004.
- [14] K. W. Morton, D. F. Mayers, *Numerical Solution of Partial Differential Equations*, wyd. 2, Cambridge University Press, 2005.
- [15] S. Williams, A. Waterman, D. Patterson, *Roofline: An Insightful Visual Performance Model for Multicore Architectures*, Communications of the ACM, 52(4), 2009, s. 65–76.
- [16] K. Datta i in., *Stencil Computation Optimization and Auto-tuning on State-of-the-Art Multicore Architectures*, Proceedings of the ACM/IEEE Conference on Supercomputing (SC'08), 2008.
- [17] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*, AFIPS Conference Proceedings, vol. 30, 1967, s. 483–485.
- [18] Y. Chen, R. Griffith, J. Liu, R. H. Katz, A. D. Joseph, *Understanding TCP Incast Throughput Collapse in Datacenter Networks*, Proceedings of the 1st ACM Workshop on Research on Enterprise Networking (WREN), 2009.
- [19] P. Luszczek, J. J. Dongarra, i in., *Introduction to the HPC Challenge Benchmark Suite*, Technical Report ICL-UT-05-01, University of Tennessee, 2005.